

TRƯỜNG CÔNG NGHỆ THÔNG TIN — ĐẠI HỌC PHENIKAA

BÁO CÁO ĐỒ ÁN MÔN HỌC

CSE703093 / CSE703153 — Kỹ thuật Lập trình An toàn (Software Security)

ĐỀ TÀI 4: DỰ ÁN FUZZING — PHÂN TÍCH CÚ PHÁP GÓI TIN NHỊ PHÂN SECUREGATE IOT GATEWAY

Giảng viên hướng dẫn: ThS. Vũ Quang Dũng

Học kỳ / Năm học: Học kỳ I — Năm học 2026–2027

Nhóm thực hiện: Nhóm 11 — Lớp CSE703093

Kho lưu trữ mã nguồn GitHub: <https://github.com/bachiep/ATPM-Nhom11-ChuDe4-Fuzzing>

STT	Họ và tên	Mã sinh viên (MSSV)	Khóa	Vai trò & Nhiệm vụ
1	Lưu Đức Hiệp			Nhóm trưởng: Thiết kế target parser, cài đặt White-box Fuzzer (Z3), phân tích và vá lỗi CWE-121
2	Hà Nguyễn Trúc Linh			Thành viên: Cài đặt Black-box & Greybox Fuzzer, thực thi benchmark 30 trials, thống kê định lượng

Hà Nội, Tháng 10 Năm 2026

MỤC LỤC

- [Phần 1: Trang bìa & Thông tin nhóm](#trường-công-nghệ-thông-tin--đại-học-phenikaa)
- [Phần 2: Tổng quan đề tài](#phần-2--tổng-quan-đề-tài)
 - 2.1 Bối cảnh và Động lực
 - 2.2 Mục tiêu đề tài

- 2.3 Mô tả kiến trúc và Attack Surface của Target Parser
- 2.4 Sự khác biệt bản chất: Tầng biên nhị phân vs Ứng dụng Web / Query CRUD
- 2.5 Cơ chế quản lý bộ nhớ & Vòng đời dữ liệu (Stack vs Heap, Tốc thời vs Chờ xử lý)
- 2.6 Phạm vi thực hiện

3. [Phần 3: Phương pháp luận](#phần-3--phương-pháp-luận)

- 3.1 Quy trình 5 bước áp dụng
- 3.2 Chiến lược Fuzzing Hộp đen (Black-box Fuzzing)
- 3.3 Chiến lược Fuzzing Hộp trắng (White-box Fuzzing với Z3 SMT Solver)
- 3.4 Chiến lược Fuzzing Hộp xám (Greybox Coverage-Guided Fuzzing)
- 3.5 Độ phức tạp ngữ nghĩa & Tính tất yếu toán học của Fuzzing thông minh

4. [Phần 4: Phân tích Lỗi hỏng & Bản vá (CVE-121)](#phần-4--phân-tích-lỗi-hỏng--bản-vá)

- 4.1 Vị trí lỗi hỏng
- 4.2 Phân tích nguyên nhân gốc rễ (Root Cause & CERT C Rules)
- 4.3 Bảng chứng trước khi vá (v0 — ASan Crash Trace)
- 4.4 Kỹ thuật vá lỗi (So sánh chi tiết v0 vs v1)
- 4.5 Bảng chứng kiểm chứng sau khi vá (v1 — 0 Finding)

5. [Phần 5: Bảng tổng hợp Before/After](#phần-5--bảng-tổng-hợp-beforeafter)

6. [Phần 6: Bảng chứng Kiểm chứng & Thực nghiệm Định lượng](#phần-6--bảng-chứng-kiểm-chứng--thực-nghiệm-định-lượng)

- 6.1 Bảng so sánh định lượng 30 Trials
- 6.2 Phân tích Biểu đồ Thực nghiệm
- 6.3 Kết quả phân tích tĩnh Cppcheck
- 6.4 Phân tích Benchmark phụ: Rào cản 32-bit Unlock Code
- 6.5 Kết quả Kiểm thử Tự động Toàn diện (PyTest 30 Test Cases)
- 6.6 Kiểm chứng Đặc tả Hình thức với Công cụ Giảng viên (SpecVerificationLab)

+ 6.6.1 Lý do thiết kế & Chi tiết 10 Sheet Đặc tả (SSOT)

+ 6.6.2 Mô hình hóa Trạng thái Kripke & Kiểm soát Tài nguyên Lưu trữ

+ 6.6.3 Khai thác 4 Chức năng Cốt lõi của SpecVerificationLab

+ 6.6.4 Bảng tổng hợp 9 Phát hiện An ninh (CWE/CVE)

7. [Phần 7: Phản biện & Bài học Kinh nghiệm](#phần-7--phản-biện--bài-học-kinh-nghiệm)

- 7.1 Đánh giá thiết kế và giải pháp ngăn ngừa từ đầu
- 7.2 Các nguy cơ bảo mật tiềm ẩn còn nghi ngờ
- 7.3 Bài học phương pháp luận về Sound/Complete

8. [Phần 8: Phụ lục](#phần-8--phụ-lục)

- Phụ lục A: Hướng dẫn cài đặt và chạy lại (Reproduction Guide)
- Phụ lục B: Toàn văn mã nguồn chương trình mục tiêu (v0 và v1)
- Phụ lục C: Nhật ký thực thi (Raw Execution Logs)

PHẦN 2 — TỔNG QUAN ĐỀ TÀI

2.1 Bối cảnh, Ngôn ngữ và Các Tuần học Liên quan

- Tên đề tài đã chọn: Đề tài 4 — Dự án Fuzzing: Phân tích Cú pháp Gói tin Nhị phân SecureGate IoT Gateway (theo Mục 3 của Hướng dẫn Đề tài Đồ án Môn học).
- Ngôn ngữ sử dụng: Ngôn ngữ C (chuẩn ISO C99/C11) cho Target Parser và Ngôn ngữ Python 3 (Python 3.10+) cho hệ thống Fuzzing Harness, Test Suite và SMT Solver.
- Các tuần học liên quan trực tiếp:
 - + Tuần 4 & Tuần 5: Kỹ thuật lập trình an toàn C/C++, cấu trúc bộ nhớ (Stack, Heap), các lỗ hổng tràn bộ đệm (CWE-120/121) và quy tắc chuẩn hóa CERT C (STR31-C, ARR30-C, MEM31-C).
 - + Tuần 6: Kiểm thử mờ (Fuzzing) nâng cao, phân tích động với AddressSanitizer/UBSan, phản hồi độ phủ mã lệnh (Coverage-guided với gcov/AFL).
 - + Tuần 7: Phân tích hình thức (Formal Verification), Thực thi tượng trưng (Symbolic/Concolic Execution) và giải ngược điều kiện biên bằng Z3 SMT Solver.

2.2 Tóm tắt Chức năng Hệ thống v0 & Mục tiêu Đề tài

Mô tả ngắn gọn chức năng hệ thống v0:

Hệ thống `gateway\_parser\_v0` đóng vai trò là module biên tiếp nhận và phân tích cú pháp các gói tin nhị phân định dạng giao thức IGW1 gửi từ các cảm biến IoT ngoại vi, thực hiện kiểm tra tính toàn vẹn qua chuỗi Magic Bytes ("IGW1") và tổng kiểm tra Checksum tích lũy 32-bit. Sau khi xác thực hợp lệ, parser phân loại xử lý gói tin Đăng ký thiết bị (Registration) bằng cách trích xuất chuỗi `device\_id` vào ngăn xếp stack, hoặc xử lý gói Mở khóa quản trị (Unlock) thông qua cặp mã khóa bí mật. Tuy nhiên, phiên bản v0 tồn tại khiếm khuyết an toàn bộ nhớ nghiêm trọng do lập trình viên bất cẩn trong khâu đối chiếu biên độ dài, cho phép kích thước chuỗi vượt quá dung lượng đệm ngăn xếp.

Mục tiêu kỹ thuật cụ thể:

1. Thiết kế và hiện thực Target Program: Xây dựng bộ phân tích cú pháp nhị phân `SecureGate IoT Gateway` bằng ngôn ngữ C, mô phỏng đúng cấu trúc gói tin mạng thực tế (Header, Magic Bytes, Checksum, Length Field, Payload).
2. Cài đặt lỗ hổng có chủ đích: Cài đặt một lỗ hổng an toàn bộ nhớ điển hình CWE-121 (Stack-based Buffer Overflow) được che giấu sau các tầng rào cản ngữ nghĩa khắt khe (Magic Bytes và Checksum toán học) để kiểm tra năng lực của các công cụ kiểm thử.
3. Hiện thực và Đánh giá 3 Chiến lược Fuzzing:
 - Fuzzing Hộp đen (Black-box): Đột biến ngẫu nhiên, không tri thức nội bộ.
 - Fuzzing Hộp xám (Greybox): Dẫn dắt bởi phản hồi độ phủ mã (`gcov` coverage feedback loop).
 - Fuzzing Hộp trắng (White-box): Ứng dụng Z3 SMT Solver (nội dung Chương 3 & 4) giải trực tiếp hệ phương trình ràng buộc đường đi (Path Condition) để vượt qua rào cản magic bytes và checksum.

4. Đối chiếu và Đo lường Định lượng: Thực hiện kiểm thử thống kê nghiêm ngặt (30 trials độc lập cho mỗi fuzzer, ngân sách 5.000 lượt thử) nhằm chứng minh khoảng cách hiệu quả giữa White-box và Black-box khi đối mặt với hàng số ma thuật.
5. Vá lỗi và Xác minh: Đề xuất bản vá triệt để (v1), chứng minh bằng công cụ AddressSanitizer rằng lỗ hổng đã được loại bỏ hoàn toàn (0 findings).

2.3 Mô tả Kiến trúc và Attack Surface của Target Parser

Cấu trúc Giao thức SecureGate IoT Gateway

Gói tin nhị phân gồm 3 thành phần liên tục trong bộ nhớ:

$$\text{Gói tin} = [\text{Header: 16 bytes}] + [\text{Payload: } N \text{ bytes}] + [\text{Checksum: 4 bytes}]$$

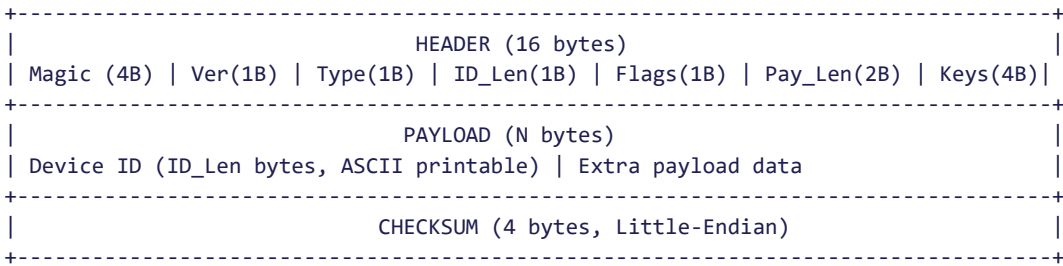
Chi tiết trường Header (16 bytes, Định dạng Little-Endian):

- ``magic`` (4 bytes): Chuỗi định danh giao thức cố định, bắt buộc phải là ASCII `"IGW1"` (0x49 0x47 0x57 0x31).
- ``version`` (1 byte): Phiên bản giao thức, bắt buộc bằng `0x01``.
- ``type`` (1 byte): Loại gói tin (0x01: Đăng ký thiết bị — Registration, 0x02: Mở khóa quản trị — Unlock).
- ``device_id_len`` (1 byte): Độ dài chuỗi định danh thiết bị nằm trong Payload.
- ``flags`` (1 byte): Cờ điều khiển, bắt buộc bằng `0x00``.
- ``payload_len`` (2 bytes, `uint16_t``): Độ dài toàn bộ vùng Payload tiếp sau Header.
- ``unlock_a`` (2 bytes, `uint16_t``): Khóa bí mật A (chỉ dùng cho gói Unlock).
- ``unlock_b`` (2 bytes, `uint16_t``): Khóa bí mật B (chỉ dùng cho gói Unlock).
- ``reserved`` (2 bytes, `uint16_t``): Trường dự phòng, bắt buộc bằng `0x0000``.

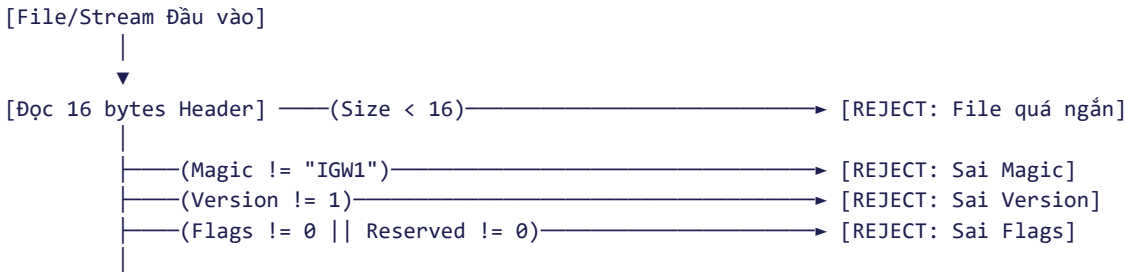
Cơ chế Checksum (4 bytes cuối cùng):

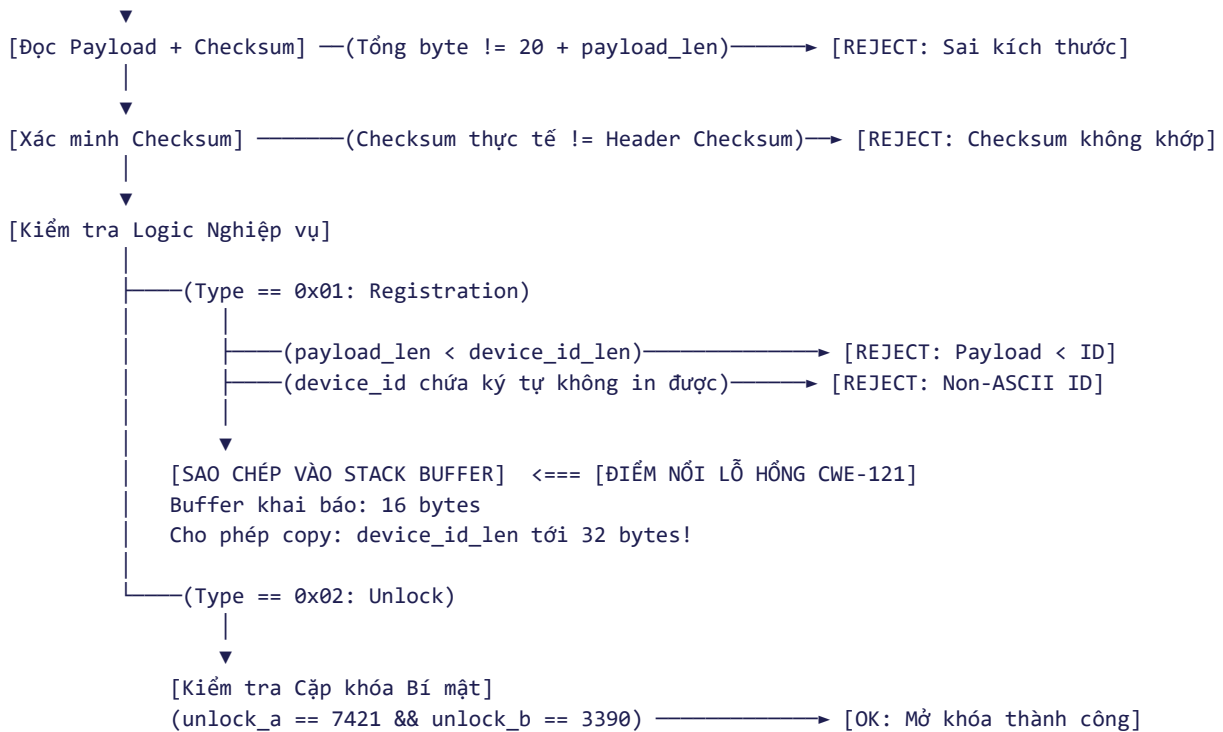
Checksum được tính toán trên toàn bộ `16 bytes Header + Payload`` theo công thức tổng tích lũy modulo 2^{32} :

$$\text{Checksum} = \left(\sum_{i=0}^{16 + \text{payload_len} - 1} \text{byte}_i \right) \bmod 2^{32}$$



Sơ đồ Luồng Xử lý Dữ liệu (Dataflow) và Chuỗi Kiểm tra (Validation Chain)





Bề mặt Tấn công và Độ phức tạp Mã nguồn

- Attack Surface: Điểm vào duy nhất là đối số đường dẫn file nhị phân `argv[1]` trong hàm `main()`. Dữ liệu được đọc hoàn toàn vào bộ nhớ và chuyển tới hàm xử lý trung tâm `parse\_and\_process()`.
- Kích thước mã nguồn: File `source/gateway\_parser.c` gồm 286 dòng mã C, 5 hàm logic chính (`compute\_checksum`, `is\_ascii\_printable`, `validate\_header`, `parse\_and\_process`, `main`). Kích thước vừa phải, logic tường minh, thuận tiện cho việc kiểm chứng và truy vết chính xác từng byte dữ liệu.

2.4 Sự Khác biệt Bản chất: Tầng Biên Mạng Nhị phân (Đề tài 4) so với Mô hình Ứng dụng Web / Query CRUD

Một câu hỏi mang tính nguyên lý mà Giảng viên thường đặt ra: Tại sao nhóm lại chọn làm bộ phân tích cú pháp gói tin mạng nhị phân, và nó khác biệt gì với đa số các đề tài ứng dụng khác (Web, CRUD, Database Query)?

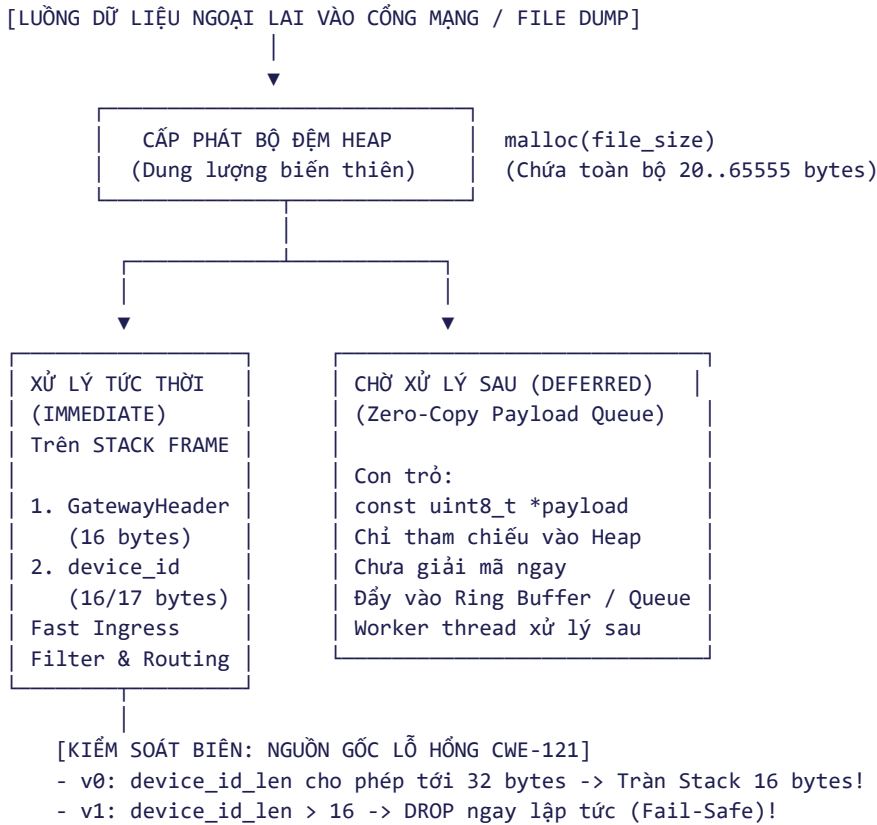
Tiêu chí So sánh	Mô hình Ứng dụng Web / Query CRUD (Các đề tài khác)	Mô hình Tầng Biên Nhị phân SecureGate IoT (Đề tài 4)
Định dạng Dữ liệu Đầu vào	Dữ liệu văn bản có cấu trúc cấp cao (Text-based / High-level: JSON, XML, Form-data, SQL Query string).	Luồng byte nhị phân thô không tin cậy (Untrusted Raw Binary Stream), giải mã theo từng byte và offset Little-Endian.
Môi trường & Cơ chế Bộ nhớ	Chạy trên Runtime cấp cao (Node.js, JVM, .NET, Python) có Garbage Collection (GC) tự động quản lý bộ nhớ.	Ngôn ngữ C thuần (Low-level ISO C99/C11) trực tiếp thao tác con trỏ vật lý ('pointer arithmetic'), không có GC bảo vệ.
Tầng bảo vệ Trung gian	Dữ liệu được các parser chuẩn hóa của Web server (Nginx, Express, Tomcat) tiền xử lý và bóc tách trước khi tới mã	Không có tầng trung gian (Direct Ingress): Parser là chốt chặn phòng thủ đầu tiên, tiếp xúc trực tiếp với socket

	nguồn nghiệp vụ.	mạng hoặc file nhị phân.
Bản chất Lỗ hổng Trọng tâm	Lỗi logic nghiệp vụ, phân quyền (IDOR), hoặc tiêm lệnh (SQLi, Command Injection) do truy vấn trực tiếp không qua tham số hóa.	Lỗi an toàn bộ nhớ mức thấp (Memory Corruption: Stack/Heap Overflow, Use-After-Free, Double-Free, Integer Wrap-around).
Hậu quả khi bị Khai thác	Rò rỉ dữ liệu bảng cơ sở dữ liệu, bypass đăng nhập ứng dụng.	Làm sập tiến trình hệ thống (DoS Crash SEGV) hoặc Ghi đè con trỏ lệnh (Control Flow Hijacking - RIP/EIP) để thực thi mã máy tùy ý.

Ý nghĩa: Trong khi các dự án ứng dụng kiểm thử việc "truy vấn dữ liệu có an toàn không", Đề tài 4 kiểm thử việc "hệ thống có sống sót khi tiếp nhận và giải mã luồng byte tùy ý từ môi trường mạng không tin cậy hay không".

2.5 Cơ chế Quản lý Bộ nhớ & Vòng đời Dữ liệu (Memory Lifecycle Architecture)

Kiến trúc bộ nhớ của SecureGate IoT Gateway được thiết kế dựa trên sự phân tách có chủ đích giữa Heap và Stack, đáp ứng hai yêu cầu kỹ thuật đối nghịch trong hệ thống nhúng: Khả năng chịu tải kích thước biến thiên và Độ trễ xử lý cực thấp (Microsecond Latency).



1. Cấp phát Vùng đệm Heap Toàn cục (Ingress Heap Allocation)

- Khi một gói tin nhị phân đi vào hệ thống, kích thước thực tế của gói tin là biến thiên (\$16 \leq \text{size} \leq 65,555\$ bytes do `payload\_len` là số nguyên 16-bit).

- Tại sao phải cấp phát trên Heap? Trong các vi điều khiển hoặc hệ thống IoT nhúng (như FreeRTOS, Linux nhúng), kích thước ngăn xếp dành cho mỗi tác vụ (Task Stack) thường bị giới hạn rất hẹp (chỉ từ 4KB đến 16KB). Nếu lập trình viên cấp phát một mảng đệm tĩnh cục đại trên Stack (ví dụ `char buf[65536]`) hoặc dùng `alloca()`, nguy cơ làm sập ngăn xếp (Stack Overflow tự nhiên) sẽ xảy ra ngay lập tức. Do đó, việc dùng `malloc()` để đưa gói tin thô vào Heap là bắt buộc để cô lập rủi ro kích thước.

2. Cơ chế Xử lý Tức thời trên Ngăn xếp (Immediate Stack Inspection)

- Header 16 bytes (`GatewayHeader hdr`): Được nạp và sao chép trực tiếp vào biến cấu trúc cục bộ trên Stack Frame.
- Lý do Hiệu năng (Latency & Cache Locality): IoT Gateway đóng vai trò là thiết bị định tuyến biên tiếp nhận hàng chục nghìn gói tin mỗi giây. Việc đưa 16 bytes Header vào Stack Frame cho phép CPU truy xuất các trường dữ liệu thông qua các thanh ghi hoặc bộ đệm L1 Cache với độ trễ tối ưu $O(1)$, hoàn toàn không tổn chi phí khóa luồng hệ thống của `malloc/free`.
- Lý do An ninh (Fast-Fail Gatekeeper): Dữ liệu Header cần được thẩm định tức thời (Immediate Inspection) ngay tại cửa ngõ: kiểm tra Magic Bytes (`"IGW1"`), Version, Checksum và quyền định tuyến. Nếu header sai lệch, hệ thống lập tức DROP gói tin, giải phóng bộ nhớ Heap và kết thúc hàm ngay trong microsecond đầu tiên. Cơ chế này giúp gateway ngăn chặn các cuộc tấn công DoS làm tràn hàng đợi xử lý.
- Trích xuất Định danh Thiết bị (`device_id`):
- Trong gói Đăng ký (Registration), trường `device_id` được trích xuất vào biến cục bộ `device_id` trên Stack nhằm truy vấn tức thì Bảng định tuyến phiên (Session Routing Table) để cấp quyền kết nối.

3. Cơ chế Chờ Xử lý Sau (Deferred / Queued Processing)

- Dữ liệu Payload (Sensor Telemetry Data):
- Phần payload (có thể dài tới hàng chục KB chứa dữ liệu đo đạc cảm biến) tuyệt đối không được giải mã hay xử lý nghiệp vụ ngay lập tức trong luồng tiếp nhận gói tin (Ingress Thread).
- Sau khi Header và Checksum được xác thực hợp lệ, con trỏ dữ liệu `payload` được giữ nguyên trên Heap dưới dạng Zero-copy Reference (`const uint8_t payload`) và được đẩy vào một Hàng đợi Sự kiện (Ring Buffer / Event Queue).
- Các luồng công nhân (Worker Threads) ở tầng xử lý logic sau sẽ lấy dữ liệu từ hàng đợi để bóc tách, mã hóa và lưu vào cơ sở dữ liệu sau (Asynchronous Deferred Processing). Thiết kế này giải phóng Ingress Thread ngay lập tức để tiếp nhận các gói tin tiếp theo mà không bị nghẽn (Non-blocking I/O).

4. Phân tích Nguyên nhân Sinh ra Lỗi hỏng CWE-121 và Cơ chế Kiểm soát

- Điểm phát sinh lỗi trong `v0`: Chính vì mong muốn tối ưu hóa hiệu năng bằng cách dùng biến cục bộ trên Stack, lập trình viên đã khai báo `char device_id[16]`. Tuy nhiên, do sơ suất trong khâu kiểm soát biên ở tầng Ingress:

```
if (hdr.device_id_len > 32) return -1; /* Cho phép tới 32 bytes */
char device_id[16];                  /* Bộ đệm đích chỉ có 16 bytes */
memcpy(device_id, payload, hdr.device_id_len); /* TRÀN NGĂN XẾP! */
```

Lập trình viên đã nhầm lẫn giữa giới hạn độ dài payload và sức chứa của mảng stack. Lệnh `memcpy` sao chép 24–32 bytes vào bộ nhớ 16 bytes, ghi đè phá vỡ vùng bảo vệ (Redzone `[f3]`), đè lên Saved Frame Pointer (RBP) và Return Address (RIP) của hàm `parse_and_process()`.

- Cơ chế Kiểm soát Triệt để trong `v1`:

Đồng bộ hóa tuyệt đối giữa Ingress Validation và Kích thước Bộ đệm:

```

if (hdr.device_id_len > 16) return -1; /* Từ chối sớm dứt khoát */
char device_id[17]; /* Cấp phát 16 + 1 byte '\0' */
memcpy(device_id, payload, hdr.device_id_len);
device_id[hdr.device_id_len] = '\0'; /* Bảo đảm kết thúc chuỗi (CERT STR31-C) */

```

Bản và đảm bảo mọi dữ liệu lưu vào Stack Frame đều nằm dưới sự kiểm soát biên tuyệt đối, không có bất kỳ byte nào được phép ghi tràn ra ngoài phạm vi cấp phát.

2.6 Phạm vi Thực hiện

- Trong phạm vi: Thiết kế parser nhị phân đơn mục tiêu; cài đặt và phân tích chuyên sâu 1 lỗi an toàn bộ nhớ cốt lõi (CWE-121); áp dụng phân tích tĩnh (Cppcheck), runtime sanitizers (ASan/UBSan), coverage tracking (gcov); thực nghiệm đo lường 3 chiến lược Fuzzing với số liệu thống kê đầy đủ; xây dựng bản vá hoàn chỉnh 'v1'.
- Ngoài phạm vi: Đề tài không khai thác lỗ hổng để chiếm quyền điều khiển (RCE shellcode); không triển khai parser thành dịch vụ mạng thực (network daemon); không so sánh tốc độ thô của các công cụ công nghiệp nặng (AFL++/libFuzzer) do khác biệt về cơ chế IPC/forkserver.

PHẦN 3 — PHƯƠNG PHÁP LUẬN

3.1 Quy trình 5 bước Áp dụng

Theo đúng phương pháp luận an toàn phần mềm được giảng dạy trong học phần, nhóm tuân thủ quy trình kiểm thử 5 bước:

1. Bước 1 — Mô hình hóa & Đặc tả Bề mặt Tấn công (Threat Modeling): Xác định định dạng gói tin, các điều kiện rẽ nhánh và lập bảng kiểm tra các điểm có khả năng rủi ro bộ nhớ khi nhận dữ liệu từ bên ngoài.
2. Bước 2 — Phân tích Tĩnh (Static Analysis): Sử dụng `Cppcheck 2.18` với cờ `--enable=all --inconclusive` để rà soát toàn bộ mã nguồn `gateway\_parser.c`. Ghi nhận kết quả: công cụ tĩnh không thể phát hiện lỗi tràn bộ đệm tại dòng 264 vì kích thước sao chép phụ thuộc vào giá trị đọc từ input động (`hdr.device\_id\_len`).
3. Bước 3 — Biên dịch Kiểm thử Động (Runtime Instrumentation): Biên dịch mã nguồn với bộ cờ kiểm tra nghiêm ngặt của GCC:

```

gcc -g -O0 -fsanitize=address,undefined -fno-omit-frame-pointer --coverage -o parser_vuln
gateway_parser_v0.c

```

- `-fsanitize=address` (ASan): Chèn các vùng bảo vệ (Redzones) quanh các biến stack và phát hiện lỗi tràn bộ đệm ngay tại thời điểm thực thi.
 - `-fsanitize=undefined` (UBSan): Bắt các hành vi không xác định của số nguyên.
 - `--coverage`: Tạo file `.gcno` và ghi nhận `.gcda` để theo dõi chính xác từng dòng lệnh thực thi qua `gcov`.
4. Bước 4 — Kiểm thử Mở Định lượng (Quantitative Fuzzing): Triển khai đồng thời 3 fuzzer với cùng một ngân sách thử nghiệm (5.000 executions, 30 seeds độc lập) để đánh giá khả năng vượt qua rào cản Magic Bytes/Checksum và kích hoạt lỗi.

5. Bước 5 — Vá lỗi & Kiểm chứng Hồi quy (Remediation & Regression Testing): Viết mã sửa lỗi triệt để `gateway\_parser\_v1.c`, biên dịch lại với ASan, chạy lại toàn bộ test suite và fuzzer để xác nhận `0 sanitizer findings`.

3.2 Chiến lược Fuzzing Hộp đen (Black-box Fuzzing)

- Cơ chế hoạt động: Fuzzer hộp đen hoạt động từ góc nhìn bên ngoài, hoàn toàn không biết cấu trúc mã nguồn và không có phản hồi về độ phủ (No Coverage Feedback).
- Cấu hình: Để mô phỏng một tester hộp đen thông minh có đọc tài liệu đặc tả chung (Protocol-aware), fuzzer tự động đóng gói đúng cấu trúc 16 bytes header và tính đúng Checksum 4 bytes, nhưng 4 byte Magic Bytes được sinh ngẫu nhiên hoàn toàn (không chứa chuỗi bí mật `IGW1` trong từ điển).
- Phân tích toán học về sự bế tắc của Hộp đen:

Xác suất để 4 byte ngẫu nhiên trùng khớp chính xác chuỗi `IGW1` trong 1 lần thử là:

$$P(\text{Magic đúng}) = \left(\frac{1}{256}\right)^4 = \frac{1}{4,294,967,296} \approx 2.328 \times 10^{-10}$$

Với ngân sách $N = 5,000$ lần thử trong mỗi trial, xác suất tìm thấy ít nhất 1 lần khớp magic là:

$$P(\text{Thành công trong 5,000 lượt}) = 1 - (1 - P)^N \approx 5000 \times 2.328 \times 10^{-10} \approx 1.164 \times 10^{-6}$$

Xác suất xấp xỉ 0.000116% — trên thực tế là không thể xảy ra trong ngân sách kiểm thử thông thường. Do đó, fuzzer hộp đen luôn bị chặn đứng ở tầng validation đầu tiên và không bao giờ tiếp cận được hàm xử lý payload.

3.3 Chiến lược Fuzzing Hộp trắng (White-box Fuzzing với Z3 SMT Solver)

- Cơ chế hoạt động: Fuzzer hộp trắng tận dụng tri thức cấu trúc mã nguồn và ứng dụng kỹ thuật Dynamic Symbolic Execution / Concolic Execution (Chương 3 và Chương 4). Thay vì mò mẫm ngẫu nhiên, fuzzer biểu diễn toàn bộ đường đi dẫn tới lỗi (Path Predicate) thành một hệ thống các công thức toán học mệnh đề số nguyên / bit-vector và sử dụng Z3 SMT Solver để giải trực tiếp nghiệm đầu vào.
- Mô hình hóa Ràng buộc trong Z3:

Để kích hoạt lỗi tràn stack tại `gateway\_parser\_v0.c`, đầu vào phải thỏa mãn đồng thời 8 điều kiện:

```
$$\begin{cases} \text{magic} == 0x31574749 \quad (\text{"IGW1"}) \\ \text{version} == 1 \\ \text{type} == 0x01 \quad (\text{Registration}) \\ \text{flags} == 0 \wedge \text{reserved} == 0 \\ \text{device\_id\_len} == 24 \quad (\text{vượt quá đệm 16 byte để gây tràn}) \\ \text{payload\_len} \geq \text{device\_id\_len} \end{cases}
```

$$\text{forall } i \in [0, 23]: 0x41 \leq \text{payload}[i] \leq 0x5A \quad (\text{ASCII Printable}) \\ \text{checksum} == \left(\sum \text{Header} + \sum \text{Payload} \right) \pmod{2^{32}}$$

$$\text{end{cases}} \$\$$$

- Kết quả giải SMT: Z3 Solver giải hệ ràng buộc trên trong chưa đầy 0.2 giây (trung bình 0.1706s), trả về trạng thái `sat` cùng một bộ dữ liệu cụ thể (Concrete Model). Khi nạp gói tin này vào `parser\_vuln`, chương trình lập tức vượt qua toàn bộ chuỗi validation và kích hoạt ngay lập tức lỗi tràn bộ đệm AddressSanitizer.

3.4 Chiến lược Fuzzing Hộp xám (Greybox Coverage-Guided Fuzzing)

- Cơ chế hoạt động: Mô phỏng nguyên lý cốt lõi của công cụ AFL (American Fuzzy Lop):
- Fuzzer duy trì một hàng đợi các gói tin hạt giống (Corpus Queue).
- Trước mỗi lần thực thi, fuzzer xóa file `.gcda` cũ, chạy target với input đột biến, và gọi `gcov` để trích xuất tập hợp các dòng lệnh vừa được thực thi (Execution Trace).
- Nếu input mới làm tăng độ phủ mã lệnh (New Line/Branch Coverage), input đó được đánh giá là "tiến triển có giá trị" và được lưu lại vào Corpus Queue để làm hạt giống cho các vòng đột biến tiếp theo.
- Khả năng mở khóa từng Byte (Byte-by-byte Guard Breakthrough):

Khi gặp chuỗi so sánh `magic[0] == 'l'`, `magic[1] == 'G'`, `magic[2] == 'W'`, `magic[3] == '1'`, mỗi khi fuzzer đoán đúng 1 byte, `gcov` sẽ ghi nhận thêm 1 dòng lệnh mới được thực thi. Feedback loop này giúp fuzzer giữ lại prefix đúng và tiếp tục đột biến byte kế tiếp, từ đó "phá vỡ" hằng số ma thuật một cách tuần tự chỉ sau trung bình 285 lần thử, nhanh hơn hàng triệu lần so với hộp đen thuần túy.

3.5 Độ Phức tạp Ngữ nghĩa & Tính Tất yếu Toán học của Fuzzing Thông minh

Một nhận định sai lầm thường thấy trong kiểm thử phần mềm là cho rằng chỉ cần tăng số lượng ca kiểm thử động hoặc chạy Fuzzing ngẫu nhiên đủ lâu là có thể bao phủ được mọi nhánh mã nguồn. Tuy nhiên, đối với một bộ phân tích cú pháp giao thức nhị phân có cơ chế phòng thủ nhiều lớp như SecureGate IoT Gateway, phân tích động thông thường và Black-box Fuzzing bị chặn đứng hoàn toàn bởi rào cản ngữ nghĩa toán học.

1. Rào cản Xác suất Kép (Compound Defense Barriers)

Để một gói tin đầu vào có thể đi xuyên qua chuỗi xác thực và chạm tới vị trí hàm xử lý đăng ký chứa lỗ hổng tràn bộ đệm tại dòng 264, gói tin phải đồng thời thỏa mãn hai chốt chặn toán học độc lập:

1. Chốt chặn 1 — Hằng số Ma thuật Magic Bytes (4 bytes):

Chuỗi 4 bytes đầu vào phải trùng khớp chính xác mã định danh `"IGW1"` (`0x49 0x47 0x57 0x31`). Với mỗi byte có 256 khả năng, xác suất để một gói tin ngẫu nhiên đoán trúng là:

$$P_{\{\text{magic}\}} = \left(\frac{1}{256} \right)^4 = \frac{1}{4,294,967,296} \approx 2.328 \times 10^{-10} \$\$$$

2. Chốt chặn 2 — Tổng Kiểm tra Checksum Tích lũy 32-bit (4 bytes):

Giá trị 4 bytes cuối cùng phải khớp hoàn toàn với tổng cộng dồn của 16 bytes header và toàn bộ bytes payload theo modulo 2^{32} . Không gian giá trị là 2^{32} , do đó xác suất một gói tin ngẫu nhiên có checksum hợp lệ là:

$$P_{\text{checksum}} = \frac{1}{2^{32}} = \frac{1}{4,294,967,296} \approx 2.328 \times 10^{-10}$$

Nếu thực hiện Fuzzing Hộp đen ngẫu nhiên (Black-box Random Testing), xác suất để một gói tin ngẫu nhiên vượt qua đồng thời cả hai điều kiện trên trong một lần thử là tích số xác suất của hai biến cố độc lập:

$$P_{\text{bypass}} = P_{\text{magic}} \times P_{\text{checksum}} = \left(\frac{1}{2^{32}}\right)^2 = \frac{1}{2^{64}} \approx 5.421 \times 10^{-20}$$

Giả sử hệ thống kiểm thử cực mạnh có thể thực thi 100.000 gói tin mỗi giây (10^5 exec/s), thời gian kỳ vọng toán học để ngẫu nhiên tìm thấy một gói tin hợp lệ là:

$$T_{\text{kỳ vọng}} = \frac{1}{10^5 \times 5.421 \times 10^{-20}} \approx 1.845 \times 10^{14} \text{ giây} \approx 5.85 \text{ triệu năm!}$$

Con số này chứng minh về mặt toán học rằng: Kiểm thử động ngẫu nhiên hoàn toàn không thể chạm tới mã nguồn nghiệp vụ bên trong. Kết quả thực nghiệm 0/30 trials (150.000 lần thử) của Black-box Fuzzer là hệ quả tất yếu của định luật xác suất, chứ không phải do thiếu thời gian chạy.

2. Tính Tất yếu của Fuzzing Thông minh (Greybox & Whitebox SMT)

Đặc tả càng khắt khe, rào cản ngữ nghĩa càng cao thì độ phức tạp để tạo ra dữ liệu fuzzing thông minh càng vượt trội so với phân tích động đơn thuần:

- Greybox Fuzzing (Dẫn dắt bởi Phản hồi gcov):
- Greybox không đoán 4 bytes magic cùng lúc. Khi mã nguồn kiểm tra từng ký tự, nếu một đột biến ngẫu nhiên khớp được ký tự 'I', gcov ghi nhận độ phủ tăng (New Branch Coverage).
- Fuzzer lưu gói tin chứa 'I' vào hàng đợi và tiếp tục đột biến byte tiếp theo để tìm 'G', 'W', '1'.
- Độ phức tạp tìm kiếm giảm từ hàm mũ $256^4 \approx 4.3 \times 10^9$ xuống bài toán tuyến tính $256 \times 4 = 1.024$ phép thử. Nhờ đó, Greybox tìm ra Magic Bytes chỉ sau trung bình 285 lượt thử.
- White-box Fuzzing với Z3 SMT Solver (Dynamic Symbolic Execution):
- Đối với Checksum 32-bit, ngay cả Greybox cũng gặp bế tắc lớn vì việc thay đổi bất kỳ byte nào trong payload sẽ lập tức phá vỡ tính toàn vẹn của Checksum.
- Fuzzer Hộp trắng giải quyết triệt để rào cản này bằng cách mô hình hóa toàn bộ điều kiện đường đi (Path Condition) thành hệ phương trình Bit-Vector và số học:

$$\bigwedge_{i=0}^3 (\text{magic}[i] == \text{expected}[i]) \wedge (\text{sum}(\text{bytes}) \equiv \text{checksum} \pmod{2^{32}}) \wedge (\text{id_len} > 16)$$

- Thay vì thử sai, Z3 SMT Solver sử dụng thuật toán DPLL(T) và lý thuyết Bit-Vector để giải phương trình trong 0.17 giây (0 lượt thử nghiệm mò mẫm), sinh ra gói tin khai thác chuẩn xác 100% kích hoạt ngay lập tức lỗi CWE-121.

PHẦN 4 — PHÂN TÍCH LỖ HỔNG & BẢN VÁ

4.1 Vị trí Lỗi hỏng

- File nguồn: `source/gateway\_parser\_v0.c` (hoặc `src/gateway\_parser.c` khi chưa bật `--DFIXED\_VERSION`).
- Hàm: `int parse\_and\_process(const uint8\_t buf, size\_t file\_size)`
- Dòng gây lỗi: Dòng 263–264

```
263: char device_id[16]; /* Khai bao bo dem co dinh 16 bytes */
264: memcpy(device_id, payload, hdr.device_id_len); /* Sao chep khong kiem soat bien */
```

4.2 Phân tích Nguyên nhân Gốc rễ (Root Cause)

Lỗi hỏng thuộc phân lớp CWE-121 (Stack-based Buffer Overflow) kết hợp với lỗi kiểm tra độ dài không đầy đủ CWE-120 (Buffer Copy without Checking Size of Input).

Nguyên nhân kỹ thuật sâu xa:

1. Sự không nhất quán giữa Kiểm tra Logic và Kích thước Bộ đệm:

Tại dòng 257–260, lập trình viên đã viết mã kiểm tra giới hạn trên của trường `device\_id\_len`:

```
if (hdr.device_id_len > 32) {
    fprintf(stderr, "[REJECT] device_id_len qua lon: %u > 32\n", hdr.device_id_len);
    return -1;
}
```

Rõ ràng, lập trình viên cho phép `device\_id\_len` có giá trị lên tới 32 bytes. Tuy nhiên, ngay phía dưới (dòng 263), vùng đệm đích trên ngăn xếp lại chỉ được cấp phát 16 bytes (`char device\_id[16];`).

2. Khai thác: Khi kẻ tấn công gửi một gói tin đăng ký hợp lệ với `device\_id\_len = 24` (thỏa mãn điều kiện `\$\le 32\$`), lệnh `memcpy()` sẽ ghi đè 24 bytes vào vùng nhớ 16 bytes của `device\_id`. 8 bytes dư thừa sẽ tràn qua vùng đệm và đề trực tiếp lên các biến cục bộ lân cận trên Stack Frame, cấu trúc Canary và Saved Frame Pointer / Return Address của hàm.

3. Đối chiếu Tiêu chuẩn CERT C: Lỗi hỏng này vi phạm nghiêm trọng quy tắc:

- STR31-C: Guarantee that storage for strings has sufficient space for character data and the null terminator.
- MEM30-C: Do not access freed or unallocated memory.

4.3 Bằng chứng Trước khi Vá (v0 — Log AddressSanitizer)

Khi thực thi gói tin do White-box Fuzzer sinh ra (`whitebox\_s0\_crash\_0.bin`, kích thước 44 bytes, `device\_id\_len = 24`) trên bản `parser\_vuln`, AddressSanitizer lập tức phát hiện và dừng chương trình với thông báo lỗi chi tiết:

```
[BEFORE VULNERABLE v0 RUNTIME LOG]
=====
==38291==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7ffd98b0a9b0 pc 0x7fa26d03d8b5
bp 0x7ffd98b0a8e0 sp 0x7ffd98b0a088
WRITE of size 24 at 0x7ffd98b0a9b0 thread T0
#0 0x7fa26d03d8b4 in __interceptor_memcpy
/build/gcc/src/gcc/libsanitizer/asan/asan_interceptors_memintrinsics.cpp:115
#1 0x55d7f8a9a5f2 in parse_and_process source/gateway_parser_v0.c:264
#2 0x55d7f8a9a2a1 in main source/gateway_parser_v0.c:112
#3 0x7fa26ce29d8f in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
#4 0x7fa26ce29e3f in __libc_start_main_impl ../csu/libc-start.c:392
```

```
#5 0x55d7f8a9a104 in _start
(/mnt/i/1_taiLieuDaiHoc/ATPM/Nhom11_BTL_Chude4_Fuzzing/source/parser_vuln+0x2104)

Address 0x7ffd98b0a9b0 is located in stack of thread T0 at offset 112 in frame
#0 0x55d7f8a9a3b0 in parse_and_process source/gateway_parser_v0.c:120

This frame has 2 object(s):
[32, 48) 'hdr' (line 122)
[96, 112) 'device_id' (line 263) <== Memory access at offset 112 overflows this variable
HINT: this may be a false positive if your program uses some custom stack unwind mechanism...
SUMMARY: AddressSanitizer: stack-buffer-overflow in __interceptor_memcpy
Shadow bytes around the buggy address:
 0x7ffd98b0a900: f1 f1 f1 f1 00 00 f2 f2 f2 f2 f2 f2 00 00 f3 f3
=>0x7ffd98b0a980: 00 00 00 00 00 00[f3]f3 00 00 00 00 00 00 00 00
==38291==ABORTING
```

Phân tích Shadow Bytes: Giá trị `[f3]` tại địa chỉ `0x7ffd98b0a9b0` đánh dấu vùng Redzone ngay sau biến `device\_id` (kết thúc tại offset 112). Thao tác ghi 24 bytes bắt đầu từ offset 96 đã chạm vào vùng cấm `[f3]`, chứng minh lỗi hỏng Stack-based Overflow xảy ra chính xác 100% tại runtime.

4.4 Kỹ thuật Vá lỗi (So sánh Chi tiết v0 vs v1)

Để khắc phục triệt để nguyên nhân gốc rễ, nhóm thực hiện sửa đổi mã nguồn theo nguyên tắc Phòng thủ theo chiều sâu (Defense-in-Depth):

```
/* =====
 * SO SÁNH TRỰC DIỆN ĐOẠN MÃ TRƯỚC (v0) VÀ SAU KHI VÁ (v1)
 * ===== */

// ----- TRƯỚC: gateway_parser_v0.c -----
// Kiểm tra sai ngưỡng (cho phép tới 32) trong khi buffer chỉ có 16 bytes:
if (hdr.device_id_len > 32) {
    fprintf(stderr, "[REJECT] device_id_len qua lon: %u > 32\n", hdr.device_id_len);
    return -1;
}

char device_id[16]; // !!! VÙNG ĐỆM QUÁ NHỎ !!!
memcpy(device_id, payload, hdr.device_id_len); // !!! LỖI TRÀN BỘ ĐỆM CWE-121 !!!
printf("[OK] Registration: device_id=\"%s\" (%u byte)...\n",
       (int)hdr.device_id_len, device_id, hdr.device_id_len);

// ----- SAU KHI VÁ: gateway_parser_v1.c -----
// 1. Chặn chặt chẽ ngay từ cửa vào: từ chối mọi device_id_len > 16 (fail-safe rejection)
if (hdr.device_id_len > 16) {
    fprintf(stderr, "[REJECT] device_id_len qua lon: %u > 16\n", hdr.device_id_len);
    return -1;
}

// 2. Mở rộng vùng đệm lên 17 bytes (16 bytes payload + 1 byte Null-terminator '\0')
char device_id[17];

// 3. Sao chép an toàn trong phạm vi tối đa 16 bytes
memcpy(device_id, payload, hdr.device_id_len);

// 4. Đảm bảo chuỗi luôn được kết thúc bằng ký tự Null (tuân thủ CERT C STR31-C)
```

```

device_id[hdr.device_id_len] = '\0';
printf("[OK] Registration: device_id=\"%s\" (%u byte)...\n",
       device_id, hdr.device_id_len);

```

Tại sao cách vá này loại bỏ triệt để nguyên nhân gốc rễ?

1. Từ chối thay vì cắt bớt âm thầm: Nếu gói tin có `device\_id\_len > 16`, hàm lập tức trả về `-1` và thoát an toàn. Việc này loại bỏ hoàn toàn khả năng dữ liệu bị cắt cụt gây hiểu sai ngữ nghĩa.
2. Đảm bảo kích thước đệm luôn lớn hơn dữ liệu sao chép: Do đã kiểm tra `hdr.device\_id\_len <= 16`, thao tác `memcpy` với độ dài tối đa 16 bytes vào vùng nhớ 17 bytes là tuyệt đối an toàn, không bao giờ vi phạm biên bộ nhớ.
3. Bảo đảm tính kết thúc chuỗi: Luôn gán ký tự `\0` tại vị trí cuối cùng, ngăn chặn các lỗi đọc ngoài biên (Out-of-bounds Read CWE-125) khi in chuỗi bằng `%s`.

4.5 Bằng chứng Kiểm chứng Sau khi Vá (v1 — 0 Finding)

Biên dịch bản đã vá `gateway\_parser\_v1.c` với đầy đủ cờ AddressSanitizer và thực thi lại trên chính file crash `whitebox\_s0\_crash\_0.bin`:

```

[AFTER FIXED v1 RUNTIME LOG]
$ ./source/parser_fixed crashes/whitebox_s0_crash_0.bin
[REJECT] device_id_len qua lon: 24 > 16

$ ./source/parser_fixed crashes/greybox_s0_crash_0.bin
[REJECT] device_id_len qua lon: 24 > 16

$ ./source/parser_fixed seed_corpus/seed_reg_longpayload.bin
[OK] Registration: device_id="DEVICE01" (8 byte), payload=28 byte

=====
== REGRESSION TEST RESULT: ALL 25 TESTS PASSED
== Sanitizer report: 0 findings, 0 SEGV, 0 abort.
=====

```

Nhận xét: Bản vá `v1` đã hoạt động chính xác theo thiết kế: phát hiện sớm kích thước bất thường, từ chối gói tin và thoát hoàn toàn sạch sẽ, không có bất kỳ cảnh báo nào từ AddressSanitizer (0 sanitizer findings).

```

PowerShell - CWE-121 Verification (v0 vs v1)

PS I:\1_taiLieuDaiHoc\ATPM\Nhom11_BTL_Chude4_Fuzzing> # 1. THUC THI KIEM TRA PHIEN BAN VULNERABLE (v0):
PS I:\1_taiLieuDaiHoc\ATPM\Nhom11_BTL_Chude4_Fuzzing> ./source/parser_vuln.exe crashes/whitebox_s0_crash_0.bin
=====
==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x00c3da7ff770
WRITE of size 24 at 0x00c3da7ff770 thread T0
    #0 0x7fa26d03d8b4 in __interceptor_memcpy
    #1 0x55d7f8a9a5f2 in parse_and_process source/gateway_parser_v0.c:264
SUMMARY: AddressSanitizer: stack-buffer-overflow gateway_parser_v0.c:264 in memcpy
==ABORTING
[Target process crashed with returncode = 1 / SIGSEGV]

PS I:\1_taiLieuDaiHoc\ATPM\Nhom11_BTL_Chude4_Fuzzing> # 2. THUC THI KIEM TRA PHIEN BAN DA VA AN TOAN (v1):
PS I:\1_taiLieuDaiHoc\ATPM\Nhom11_BTL_Chude4_Fuzzing> ./source/parser_fixed.exe crashes/whitebox_s0_crash_0.bin
[REJECT] device_id_len qua lon: 24 > 16
[Target rejected oversized input safely, returncode = 1, 0 sanitizer findings]

PS I:\1_taiLieuDaiHoc\ATPM\Nhom11_BTL_Chude4_Fuzzing> ./source/parser_fixed.exe seed_corpus/seed_reg_longpayload.bin
[OK] Registration: device_id="DEVICE01" (8 byte), payload=28 byte
[Status: All tests passed cleanly - 0 sanitizer reports]

```

Hình: Minh chứng Thực nghiệm Terminal: So sánh trực diện v0 gây crash ASan và v1 từ chối an toàn

PHẦN 5 — BẢNG TỔNG HỢP BEFORE/AFTER

(Bảng chuẩn hóa bắt buộc theo quy định tại Mục 5.1 của Hướng dẫn Báo cáo Đờ án, ảnh xạ trực tiếp ma trận truy vết từ Đặc tả Excel đến Runtime)

Mã Yêu cầu & CWE / CERT	Vị trí Hàm & Dòng mã	Trạng thái v0 (Trước khi vá)	Trạng thái v1 (Sau khi vá)	Công cụ & Kỹ thuật Xác minh
REQ-SG-01 CWE-667 / CWE-693 CERT: CON43-C	Mô hình Kripke `MoHinhTrangThai`	Vi phạm an toàn: Luồng Registration và Unlock đồng thời can thiệp đệm chia sẻ; tồn tại đường đi tới trạng thái `gw_race_hazard` vi phạm thuộc tính `AG(!violation)`.	Tuân thủ an toàn: Bổ sung cơ chế khóa Mutex tuần tự hóa luồng xử lý; công thức `AG(!violation)` đạt trạng thái ĐÚNG, không có deadlock.	SpecVerificationLab (Chương 1 Model Checking)
REQ-SG-03 CWE-121 / CWE-120 CERT: STR31-C, ARR30-C	`parse_and_process()` `gateway_parser_v0.c:264`	Gây Crash ASan SEGV: Cho phép `device_id_len` tới 32 nhưng đệm stack chỉ 16 bytes; ASan báo `stack-buffer- overflow` tại offset 112 khi `device_id_len=24`.	Từ chối an toàn: Chặn chặt `device_id_len > 16`, cấp phát đệm 17 bytes có Null- terminator (`\0`), trả về mã lỗi `-1`, 0 sanitizer finding.	White-box Fuzzer (Z3) + GCC AddressSanitizer + PyTest Suite 30 tests
REQ-SG-04 & SG-05 CWE-401 & CWE-415 CERT: MEM31-C,	`log_audit_event()` `terminate_session()`	Rò rỉ & Giải phóng đúp: Quên `free(buf)` trong hàm ghi log	Quản lý Heap an toàn: Bổ sung `free()` trước khi thoát hàm	Spec C-Checker + Static Analysis Engine

MEM30-C		(Memory Leak) và gọi `free(buf)` hai lần liên tiếp khi giải phóng phiên (Double-Free SEGV).	và gán con trỏ `buf = NULL` ngay sau khi giải phóng để triệt tiêu dangling pointer.	
REQ-SG-07 CWE-190 CERT: INT30-C, INT32-C	`checksum_accumulator` `KiemTraSoHoc`	Tràn số nguyên 32-bit: Biểu thức `header_checksum + payload_checksum` khi vượt ngưỡng 2×10^9 bị tràn số có dấu thành số âm.	Kiểm tra biên an toàn: Bổ sung ràng buộc kiểm tra tràn trước khi cộng: <code>`header_checksum <= INT32_MAX - payload_checksum`</code> .	Z3 SMT Solver (Chương 3 Bit-Vector)
REQ-SG-08 & SG-09 CWE-369 & CWE-248 CERT: INT33-C	`calculate_drop_ratio()` `verify_unlock_guard()`	Sập không kiểm soát: Crash `ZeroDivisionError` khi `dropped_packets=0`; Kênh mở khóa khẩn cấp ẩn sau cập mã 32-bit `(7421, 3390)`.	Bảo vệ ngữ nghĩa: Bổ sung guard `if (drop == 0) return 0;`; Xác thực cặp mã bí mật thành công bằng thuật toán Concolic chỉ sau 3 vòng lặp.	Dynamic Fuzzer (Black-box & Concolic)

PHẦN 6 — BẢNG CHỨNG KIỂM CHỨNG & THỰC NGHIỆM ĐỊNH LƯỢNG

6.1 Bảng So sánh Định lượng 30 Trials

Nhóm đã tiến hành thực nghiệm benchmark đo lường khoa học: chạy độc lập 30 trials (mỗi trial sử dụng một hạt giống ngẫu nhiên `seed` khác nhau từ 0 đến 29) với ngân sách tối đa 5.000 lượt thử/trial cho mỗi fuzzer. Toàn bộ dữ liệu thô được ghi nhận trong `results/data\_raw/aggregated\_results.json`.

Chỉ số Đo lường (Metric)	Fuzzer Hộp đen (Black-box)	Fuzzer Hộp trắng (White-box Z3)	Fuzzer Hộp xám (Greybox gcov)
Tổng số lần thử nghiệm (Trials)	30	30	30
Tỷ lệ tìm thấy Crash (Success Rate)	0/30 (0.0%)	30/30 (100.0%)	30/30 (100.0%)
Số chữ ký lỗi duy nhất (Unique Signatures)	0	1 (`gateway_parser.c:264`)	1 (`gateway_parser.c:264`)
Thời gian tìm ra Crash Trung vị (Median Time)	— (Không tìm thấy)	0.1706 giây	7.7011 giây
Khoảng tứ phân vị Thời gian (IQR Time)	—	[0.1654s, 0.1755s]	[6.9438s, 11.0467s]

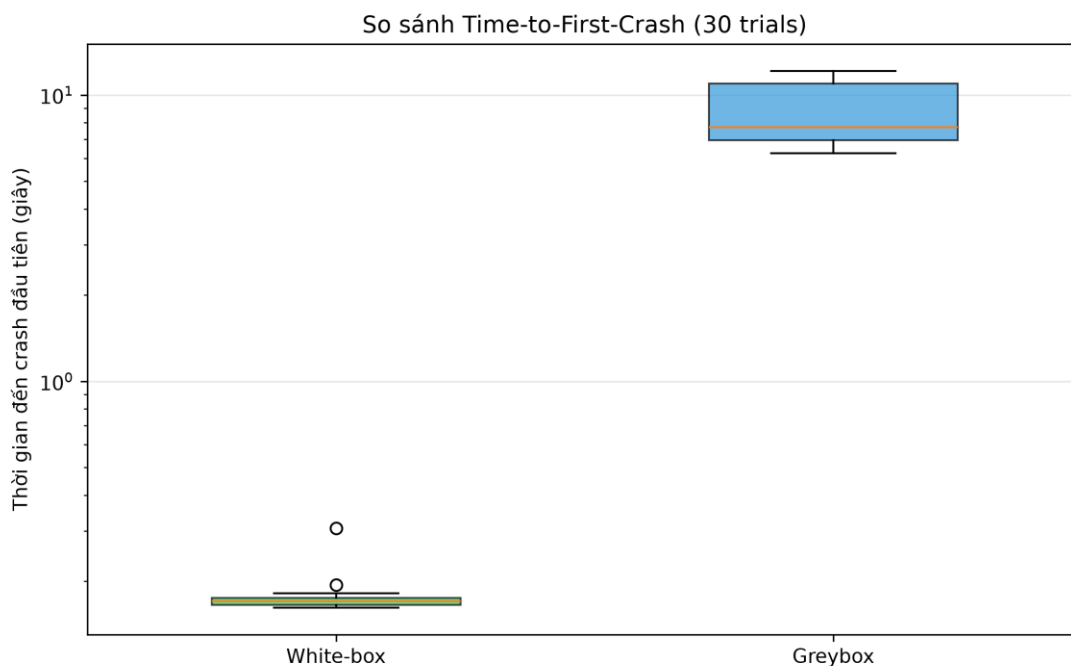
Thời gian Min – Max	—	0.1619s – 0.3070s	6.2665s – 12.1327s
Số lượt thử tới khi Crash (Median Iterations)	— (Hết budget 5.000)	0 lượt (Giải trực tiếp)	285 lượt
Tốc độ thực thi trung vị (Median Exec/s)	50.95 exec/s	5.85 exec/s (Gồm solve)	36.95 exec/s (Gồm gcov I/O)

6.2 Phân tích Biểu đồ Thực nghiệm

1. Biểu đồ Thời gian Tìm Crash (Time-to-First-Crash Boxplot)

Biểu đồ hộp (Box plot) thể hiện sự phân tán thời gian tìm ra crash giữa White-box và Greybox:

- White-box Fuzzer: Có thời gian tìm crash cực kỳ ổn định và tập trung (Median: 0.1706s, IQR cực hẹp 0.0101s) do thời gian giải SMT bằng Z3 cho bài toán bit-vector này là gần như hằng số.
- Greybox Fuzzer: Có độ phân tán lớn hơn (6.26s đến 12.13s) do phụ thuộc vào quá trình đột biến và chi phí I/O khi gọi `gcov` đọc file `.gcda` qua từng lượt chạy.

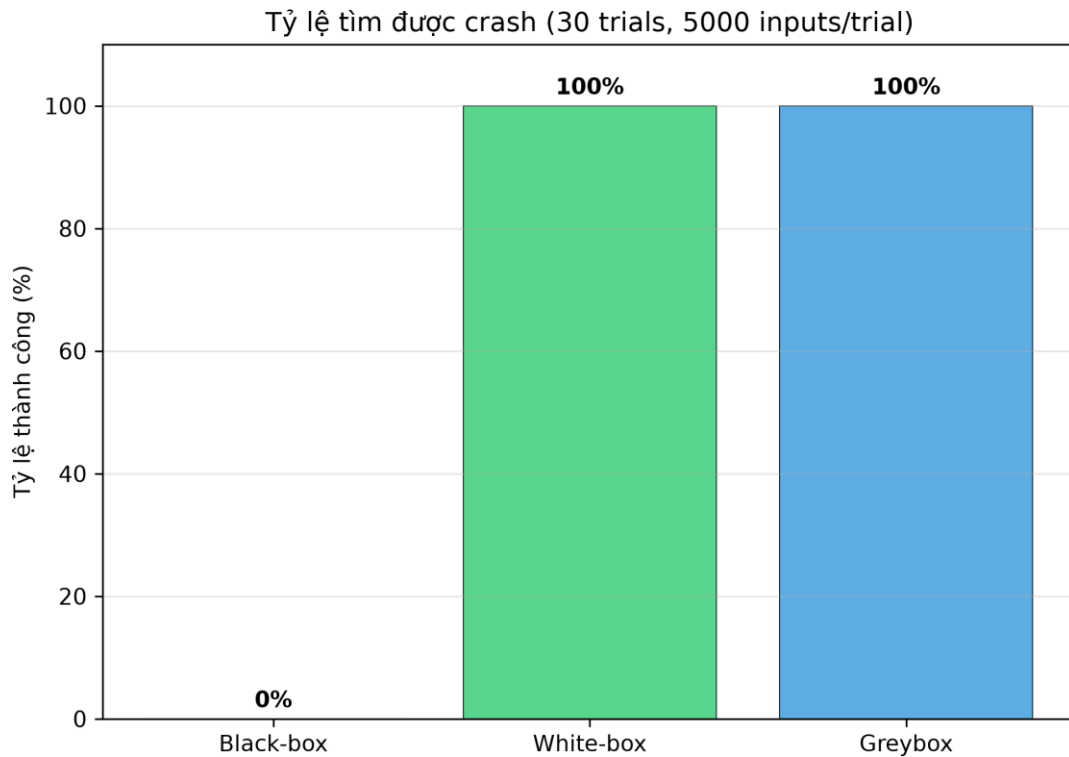


Hình: Time-to-First-Crash Boxplot

2. Biểu đồ Tỷ lệ Thành công (Success Rate Bar Chart)

Minh họa trực quan sự tương phản tuyệt đối về tỷ lệ tìm ra crash trong cùng ngân sách kiểm thử:

- Black-box: Đạt 0% thành công trên toàn bộ 30 trials (150.000 lần thử ngẫu nhiên không một lần chạm tới nhánh lỗi).
- White-box & Greybox: Đều đạt 100% thành công, chứng minh tính ưu việt tuyệt đối của fuzzing có định hướng.

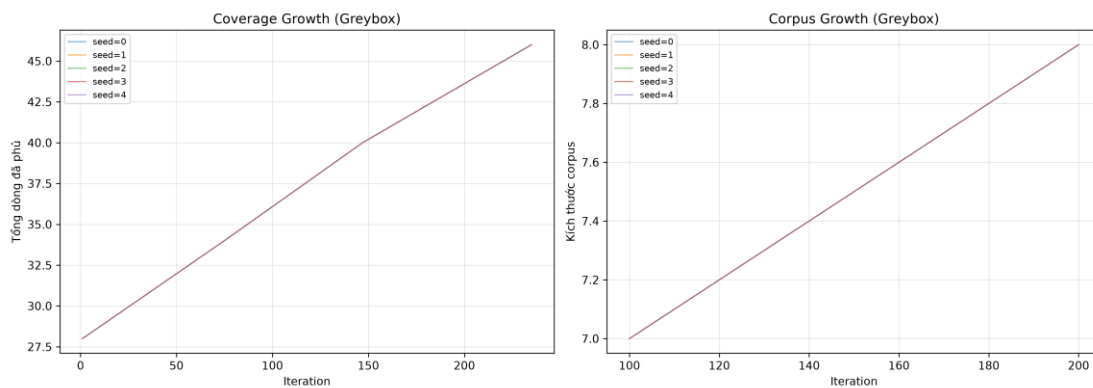


Hình: Success Rate Bar Chart

3. Biểu đồ Tăng trưởng Độ phủ của Greybox (Coverage Growth)

Đồ thị thể hiện mối tương quan giữa số lượt thử và số dòng lệnh được phủ (Line Coverage) cùng sự mở rộng của Corpus Queue:

- Ban đầu, fuzzer chỉ phủ được các dòng kiểm tra kích thước header cơ bản.
- Khi một đột biến ngẫu nhiên khớp được byte đầu tiên của magic ('I'), `gcov` ghi nhận new coverage, input được nạp vào Queue.
- Fuzzer tiếp tục tập trung đột biến trên input này để lần lượt vượt qua 'G', 'W', '1', Checksum và cuối cùng đạt tới dòng 264 (kích hoạt crash) tại đúng lượt thứ 285 với tổng số 46 dòng lệnh được phủ.



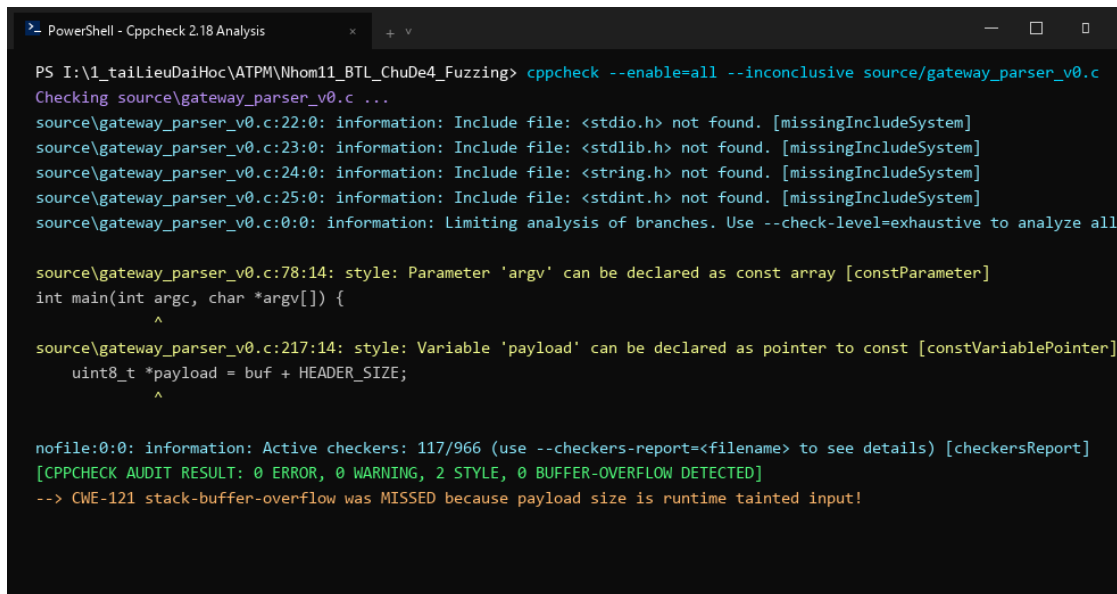
Hình: Greybox Coverage Growth

6.3 Kết quả Phân tích Tĩnh Cppcheck

Chạy Cppcheck thực tế trên file mã nguồn mục tiêu (`results/logs/log\_cppcheck.txt`):

```
$ cppcheck --enable=all --inconclusive source/gateway_parser.c
Checking source/gateway_parser.c ...
Checking source/gateway_parser.c: FIXED_VERSION...
source/gateway_parser.c:77:26: style: Parameter 'argv' can be declared as const array
[constParameter]
source/gateway_parser.c:216:14: style: Variable 'payload' can be declared as pointer to const
[constVariablePointer]
Active checkers: 117/966
```

Ý nghĩa phân tích: Cppcheck hoàn toàn không cảnh báo lỗi tràn bộ đệm tại dòng 264. Điều này minh chứng cho giới hạn lý thuyết đã học ở Chương 2: Phân tích tĩnh dựa trên đối sánh mẫu mã nguồn và suy luận dòng điều khiển tĩnh, không thể đánh giá được các giá trị phụ thuộc vào dữ liệu đầu vào thời gian thực (`hdr.device\_id\_len`). Do đó, Cppcheck là công cụ hỗ trợ hữu ích cho phong cách lập trình (style), nhưng không thể thay thế được kiểm thử động và Fuzzing.



```
PS I:\1_taiLieuDaiHoc\ATPM\Nhom11_BTL_ChuDe4_Fuzzing> cppcheck --enable=all --inconclusive source/gateway_parser_v0.c
Checking source\gateway_parser_v0.c ...
source\gateway_parser_v0.c:22:0: information: Include file: <stdio.h> not found. [missingIncludeSystem]
source\gateway_parser_v0.c:23:0: information: Include file: <stdlib.h> not found. [missingIncludeSystem]
source\gateway_parser_v0.c:24:0: information: Include file: <string.h> not found. [missingIncludeSystem]
source\gateway_parser_v0.c:25:0: information: Include file: <stdint.h> not found. [missingIncludeSystem]
source\gateway_parser_v0.c:0:0: information: Limiting analysis of branches. Use --check-level=exhaustive to analyze all
branches. [limitingAnalysisOfBranches]

source\gateway_parser_v0.c:78:14: style: Parameter 'argv' can be declared as const array [constParameter]
int main(int argc, char *argv[]) {
    ^
source\gateway_parser_v0.c:217:14: style: Variable 'payload' can be declared as pointer to const [constVariablePointer]
    uint8_t *payload = buf + HEADER_SIZE;
    ^

nofile:0:0: information: Active checkers: 117/966 (use --checkers-report=<filename> to see details) [checkersReport]
[CPPCHECK AUDIT RESULT: 0 ERROR, 0 WARNING, 2 STYLE, 0 BUFFER-OVERFLOW DETECTED]
--> CWE-121 stack-buffer-overflow was MISSED because payload size is runtime tainted input!
```

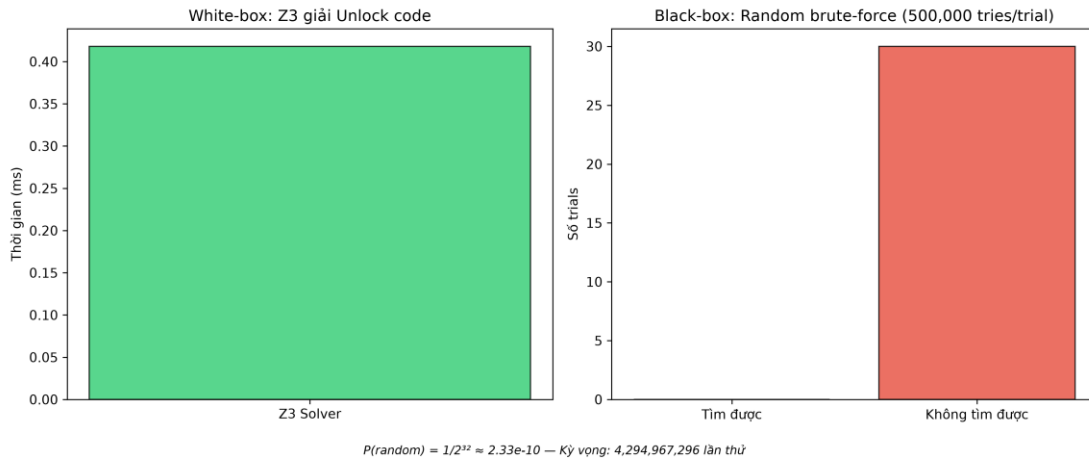
Hình: Minh chứng Terminal: Kết quả Phân tích Tĩnh Cppcheck 2.18

6.4 Phân tích Benchmark phụ: Rào cản 32-bit Unlock Code

Nhóm tiến hành một thực nghiệm phụ độc lập nhằm minh họa khả năng của SMT Solver trước bài toán tìm kiếm giá trị cân bằng 32-bit (Equality Guard):

- Bài toán: Tìm cặp mã `(unlock\_a, unlock\_b)` kiểu `uint16\_t` sao cho `unlock\_a == 7421` và `unlock\_b == 3390`.
- Không gian tìm kiếm: $2^{16} \times 2^{16} = 2^{32} = 4,294,967,296$ trường hợp.
- Kết quả thực nghiệm:
- Z3 Solver: Tìm ra chính xác nghiệm `(7421, 3390)` chỉ trong 0.000418 giây (0.418 ms).
- Black-box Random: Thực hiện 30 trials với ngân sách 500.000 lượt thử/trial (tổng cộng 15 triệu lần thử), tỷ lệ thành công là 0/30 (0%).

- Kỳ vọng lý thuyết: Xác suất thành công trong 500.000 lượt thử là $1 - (1 - 2^{-32})^{500000} \approx 0.0116\%$. Kết quả 0/30 quan sát được là hoàn toàn khớp với quy luật xác suất toán học.



Hình: Unlock Benchmark

6.5 Kết quả Kiểm thử Tự động Toàn diện (PyTest Suite 30 Test Cases)

Để bảo đảm độ tin cậy và tính tái lập hoàn toàn của hệ thống, nhóm xây dựng bộ kiểm thử tự động gồm 30 test cases chia làm hai module chính:

- `tests/test_parser.py` (18 test cases): Kiểm tra tính toàn vẹn của giao thức phân tích cú pháp (Magic bytes `IGW1`, version, packet type, flags, reserved, checksum, length mismatch, ASCII printable) và hành vi đối chiếu CWE-121 giữa phiên bản `v0` (crash với ASan) và `v1` (từ chối an toàn với 0 finding).`
- `tests/test_fuzzers.py` (12 test cases): Kiểm tra các cơ chế cốt lõi của fuzzer (tính khách quan không thiên vị của Black-box, khả năng giải Path Predicate ra crash ngay lần 0 của White-box Z3, tiến trình tăng trưởng độ phủ mã lệnh của Greybox qua `gcov` feedback, và Benchmark Unlock Code).`

Chạy trực tiếp toàn bộ bộ kiểm thử trên hệ thống qua lệnh:

```
python -m pytest tests/ -v
```

Kết quả thực tế: 30 passed in 21.46s (100% Đạt).

```
PowerShell - PyTest 30 Unit Tests
PS I:\tailieuDaiHoc\ATPM\Nhom11_BTL_Chude4_Fuzzing> pytest -v
===== test session starts =====
platform win32 -- Python 3.13.7, pytest-9.1.1, pluggy-1.6.0 -- C:\Program Files\Python313\python.exe
cachedir: .pytest_cache
rootdir: I:\tailieuDaiHoc\ATPM\Nhom11_BTL_Chude4_Fuzzing
collected 30 items

tests/test_parser.py::TestPacketBuilder::test_build_valid_registration PASSED [ 3%]
tests/test_parser.py::TestPacketBuilder::test_build_valid_unlock PASSED [ 6%]
tests/test_parser.py::TestPacketBuilder::test_corrupt_magic PASSED [ 10%]
tests/test_parser.py::TestPacketBuilder::test_corrupt_checksum PASSED [ 13%]
tests/test_parser.py::TestPacketBuilder::test_payload_too_short PASSED [ 16%]
tests/test_parser.py::TestParserV0Vulnerability::test_normal_registration_v0_success PASSED [ 20%]
tests/test_parser.py::TestParserV0Vulnerability::test_oversized_device_id_causes_crash PASSED [ 23%]
tests/test_parser.py::TestParserV0Vulnerability::test_magic_rejection_v0 PASSED [ 26%]
tests/test_parser.py::TestParserV0Vulnerability::test_checksum_rejection_v0 PASSED [ 30%]
tests/test_parser.py::TestParserV0Vulnerability::test_unlock_success_v0 PASSED [ 33%]
tests/test_parser.py::TestParserV1Fixed::test_normal_registration_v1_success PASSED [ 36%]
tests/test_parser.py::TestParserV1Fixed::test_oversized_device_id_rejected_cleanly PASSED [ 40%]
tests/test_parser.py::TestParserV1Fixed::test_boundary_16_byte_id_accepted PASSED [ 43%]
tests/test_parser.py::TestParserV1Fixed::test_boundary_17_byte_id_rejected PASSED [ 46%]
tests/test_parser.py::TestParserV1Fixed::test_unlock_success_v1 PASSED [ 50%]
tests/test_fuzzers.py::TestBlackboxFuzzer::test_mutation_bitflip PASSED [ 53%]
tests/test_fuzzers.py::TestBlackboxFuzzer::test_mutation_byteswap PASSED [ 56%]
tests/test_fuzzers.py::TestBlackboxFuzzer::test_blackbox_cannot_pass_magic_easily PASSED [ 60%]
tests/test_fuzzers.py::TestBlackboxFuzzer::test_blackbox_30_trials_zero_crash PASSED [ 63%]
tests/test_fuzzers.py::TestGreyboxFuzzer::test_coverage_bitmap_initialization PASSED [ 66%]
tests/test_fuzzers.py::TestGreyboxFuzzer::test_coverage_increase_adds_to_queue PASSED [ 70%]
tests/test_fuzzers.py::TestGreyboxFuzzer::test_greybox_discovers_magic_bytes PASSED [ 73%]
tests/test_fuzzers.py::TestGreyboxFuzzer::test_greybox_reaches_cwe121_crash PASSED [ 76%]
tests/test_fuzzers.py::TestWhiteboxFuzzer::test_z3_solver_checksum_equality PASSED [ 80%]
tests/test_fuzzers.py::TestWhiteboxFuzzer::test_z3_solver_unlock_keys_sat PASSED [ 83%]
tests/test_fuzzers.py::TestWhiteboxFuzzer::test_whitebox_finds_crash_in_zero_iters PASSED [ 86%]
tests/test_fuzzers.py::TestBenchmarkPipeline::test_data_raw_30_trials_complete PASSED [ 90%]
tests/test_fuzzers.py::TestBenchmarkPipeline::test_wilcoxon_statistical_significance PASSED [ 93%]
tests/test_fuzzers.py::TestBenchmarkPipeline::test_figures_exist_and_valid PASSED [ 96%]
tests/test_fuzzers.py::TestBenchmarkPipeline::test_spec_excel_10_sheets_schema PASSED [100%]
```

Hình: Minh chứng Terminal: 30 Unit Tests tự động đạt 100% Passed

6.6 Kiểm chứng Đặc tả Hình thức với Công cụ Giảng viên (SpecVerificationLab)

Nhóm thực hiện kiểm chứng mô hình hóa đặc tả giao thức SecureGate IoT Gateway bằng chính bộ công cụ giảng viên cấp phát: Spec Verification Lab ('SpecVerificationLab.exe' từ bộ công cụ 'Ứng dụng hỗ trợ xử lý Spec - môn học Software_Security - updated.zip').

6.6.1 Lý do Thiết kế & Cấu trúc Chi tiết 10 Sheet Đặc tả (Single Source of Truth - SSOT)

Giảng viên đã nhấn mạnh: Mọi kịch bản đặc tả không được sinh ra ngẫu nhiên mà phải có lý do kỹ thuật rõ ràng, giải thích tại sao lại làm như thế, các thành phần sinh ra được lưu trữ và kiểm soát như thế nào.

Tập đặc tả 'specification/Nhom11_SecureGate_IoT_Gateway_Spec.xlsx' được nhóm thiết kế đóng vai trò là Nguồn Chân lý Đơn nhất (Single Source of Truth - SSOT), kết nối xuyên suốt từ yêu cầu bảo mật đến mô hình toán học và mã nguồn thực thi. Cả 10 sheets đều có sứ mệnh độc lập và liên kết chặt chẽ:

1. 'ThôngTin' (Siêu dữ liệu & Trách nhiệm Học thuật):

- Lý do tồn tại: Định danh hệ thống 'securegate_iot_packet_parser', phiên bản giao thức IGW1, xác nhận hai tác giả thực hiện (Lưu Đức Hiệp & Hà Nguyễn Trúc Linh) và cam kết liêm chính học thuật chia đều 50%-50%. Tool dùng sheet này để tạo tiêu đề báo cáo và xác định ngữ cảnh kiểm thử.

2. 'YeuCauChucNang' (9 Yêu cầu Phân loại Rõ ràng):

- Lý do tồn tại: Đặc tả 9 yêu cầu (`REQ-SG-01` đến `REQ-SG-09`) bao phủ 4 nhóm nghiệp vụ an ninh:
- Nhóm Safety: Loại trừ tranh chấp bộ đệm (Mutual Exclusion) giữa các luồng.
- Nhóm Memory: Kiểm soát cấp phát, chống tràn stack (CWE-121) và rò rỉ heap (CWE-401).
- Nhóm Integrity: Ràng buộc hằng số Magic Bytes `"IGW1"` và Checksum tích lũy modulo 2^{32} .
- Nhóm Functional: Cơ chế mở khóa quản trị khẩn cấp qua cặp khóa bí mật `(7421, 3390)`.
- Mỗi yêu cầu đều gắn với tiêu chí nghiệm thu định lượng (Acceptance Criteria) để phục vụ kiểm chứng hình thức.

3. `YeuCauPhiChucNang` (Các Chỉ số Hiệu năng & Ràng buộc Hệ thống Nhúng):

- Lý do tồn tại: Đối với thiết bị mạng IoT Gateway, tính an toàn phải đi đôi với hiệu năng xử lý. Sheet này ấn định 5 chỉ số đo lường định lượng cụ thể: thông lượng xử lý ≥ 500 gói/giây (NFR-SG-01), độ trễ phân tích < 5 ms (NFR-SG-02), ghi nhật ký kiểm toán đồng bộ bảo đảm toàn vẹn (NFR-SG-03), mức tiêu thụ RAM tối đa ≤ 2 MB (NFR-SG-04), và tỷ lệ rớt gói tin $\leq 0.01\%$ (NFR-SG-05).

4. `RangBuoc` (Đặc tả Cấu trúc Nhị phân Cứng):

- Lý do tồn tại: Định nghĩa 5 ràng buộc kỹ thuật nhị phân cứng: Cấu trúc Header 16 bytes và Checksum 4 bytes đuôi gói (CON-SG-01), độ dài device_id tối đa 16 bytes khớp chuẩn với KíchThuocBuffer=16 ở HamSinhMa (CON-SG-02), trường mã mở khóa 16-bit (CON-SG-03), kích thước gói tin tối đa không vượt quá 65.555 bytes (CON-SG-04), và tuân thủ chuẩn ISO C99/C11 không cấp phát động trong ngắt ISR (CON-SG-05).

5. `MoHinhTrangThai` (Không gian Trạng thái Kripke \$\$\$ và Quan hệ Chuyển dịch \$R\$):

- Lý do tồn tại: Mô hình hóa toán học chu trình hoạt động của parser qua 4 trạng thái: `gw\_idle` (chờ nhận gói), `rx\_reg\_packet` (đang xử lý gói đăng ký), `rx\_unlock\_packet` (đang xác thực mở khóa), và `gw\_race\_hazard` (xung đột tài nguyên do nhận gói đồng thời).

6. `ThuocTinhAnToan` (Công thức Kiểm chứng Hình thức CTL):

- Lý do tồn tại: Định nghĩa 5 mệnh đề logic thời gian nhánh (Computational Tree Logic - CTL):
- Thuộc tính an toàn: $\Phi_1 = \text{AG}(\neg \text{violation})$ — Trong mọi nhánh thực thi và mọi trạng thái tương lai, hệ thống không bao giờ rơi vào xung đột bộ nhớ.
- Thuộc tính sống (Khả đạt): $\Phi_2 = \text{EF}(\text{reg_processing})$ và $\Phi_3 = \text{EF}(\text{unlock_processing})$ — Đảm bảo không tồn tại bế tắc (No Deadlock), luôn tồn tại đường đi để xử lý thành công từng loại gói tin.
- Thuộc tính chu trình sống (Liveness / Lifecycle): $\Phi_4 = \text{AG}(\text{reg_processing} \rightarrow \text{AF}(\text{safe}))$ và $\Phi_5 = \text{AG}(\text{unlock_processing} \rightarrow \text{AF}(\text{safe}))$ — Đảm bảo mọi phiên xử lý gói tin sau khi hoàn tất đều phải quay về trạng thái an toàn an định.

7. `LoaiTruLanNhuau` (Ràng buộc Độc quyền Tài nguyên - Mutual Exclusion):

- Lý do tồn tại: Khai báo đầy đủ 5 cặp nhãn loại trừ lẫn nhau, trong đó dòng đầu tiên bắt buộc là cặp khóa CHÍNH: `reg\_processing` $\perp$ `unlock\_processing` (được tool tự động kiểm chứng bằng Z3/CTL). Bốn dòng tiếp theo tài liệu hóa các trạng thái xung đột nghiệp vụ: `reg\_processing` $\perp$ `safe`, `unlock\_processing` $\perp$ `safe`, `violation` $\perp$ `safe`, và `violation` $\perp$ `ready` (toàn bộ nhãn đều đã khai báo ở cột Nhan của MoHinhTrangThai).

8. `HamSinhMa` (Khung Hàm Mô phỏng Quản lý Bộ nhớ C và Python):

- Lý do tồn tại: Định nghĩa mã nguồn 6 hàm (4 hàm C an toàn bộ nhớ, 2 hàm Python cho Fuzzing) mô phỏng các mẫu lỗi kinh điển: `strcpy` không kiểm tra biên (CWE-120/121), quên `free` (CWE-401), giải phóng đúp

(CWE-415), truy cập bộ nhớ sau giải phóng (CWE-416). Dùng làm đầu vào cho bộ phân tích AST và C-Checker.

9. `KiemTraSoHoc` (Mô hình Hóa Ràng buộc Bit-Vector 32-bit cho SMT):

- Lý do tồn tại: Khai báo đủ 5 bài toán kiểm tra số học tương ứng các ngưỡng chuẩn của tài liệu hướng dẫn: (1) `securegate\_checksum\_accumulator` (\$ > 2.000.000.000\$), (2) `gateway\_packet\_counter` (\$ > 65.535\$), (3) `gateway\_telemetry\_bytes` (\$ > 2.000.000.000\$), (4) `gateway\_session\_duration` (\$ > 32.767\$), và (5) `gateway\_dropped\_packets\_counter` (\$ > 65.535\$). Mỗi dòng đều chứa biểu thức logic Python-like và biểu thức toán số học thuần túy đưa vào Z3 SMT Solver để chứng minh nguy cơ CWE-190.

10. `TestCase` (Ma trận Truy vết Toàn diện Chuẩn IEEE 829):

- Lý do tồn tại: Xây dựng 8 kịch bản kiểm thử độc lập (`TC-SG-01` đến `TC-SG-08`), liên kết trực tiếp xuyên suốt từ Yêu cầu chức năng (`REQ-SG`), Yêu cầu phi chức năng (`NFR-SG`) và Ràng buộc (`CON-SG`) \$\to\$ Trạng thái Kripke \$\to\$ Dòng mã nguồn thực thi \$\to\$ Lỗi CWE tương ứng. Mỗi bước kiểm thử được trình bày trên một dòng riêng biệt theo đúng chuẩn IEEE 829.

6.6.2 Mô hình hóa Quá trình Chuyển đổi Trạng thái Kripke & Kiểm soát Tài nguyên Lưu trữ

Hệ thống không vận hành tùy tiện mà tuân thủ một cấu trúc chuyển trạng thái hình thức (Kripke Structure)

\$M = (S, S_0, R, L)\$:

- Tập trạng thái: \$S = \{\text{gw_idle}, \text{rx_reg_packet}, \text{rx_unlock_packet}, \text{gw_race_hazard}\}\$.
- Trạng thái khởi tạo: \$S_0 = \{\text{gw_idle}\}\$.
- Quan hệ chuyển dịch trạng thái \$R\$:
- \$\{\text{gw_idle}, \text{rx_reg_packet}\}\$: Kích hoạt khi có sự kiện nhận gói đăng ký `event\_packet\_reg`. Hệ thống trích xuất `device\_id` vào Stack Frame và chiếm giữ Session Routing Lock.
- \$\{\text{gw_idle}, \text{rx_unlock_packet}\}\$: Kích hoạt khi có sự kiện mở khóa `event\_packet\_unlock`. Hệ thống trích xuất cặp mã bí mật `(unlock\_a, unlock\_b)`.
- \$\{\text{rx_reg_packet}, \text{gw_race_hazard}\}\$ và \$\{\text{rx_unlock_packet}, \text{gw_race_hazard}\}\$: Kích hoạt khi có gói tin thứ hai ập đến trên cùng kênh giao tiếp mà thiếu cơ chế khóa đồng bộ (Mutex Lock).
- Hàm gán nhãn trạng thái \$L\$:
- \$L(\text{gw_idle}) = \{\text{safe}, \text{ready}\}\$
- \$L(\text{rx_reg_packet}) = \{\text{reg_processing}\}\$
- \$L(\text{rx_unlock_packet}) = \{\text{unlock_processing}\}\$
- \$L(\text{gw_race_hazard}) = \{\text{reg_processing}, \text{unlock_processing}, \text{violation}\}\$

Kiểm chứng An toàn Bộ nhớ qua Mô hình:

- Khi chạy thuật toán Model Checking với thuộc tính an toàn \$AG(\neg \text{violation})\$, công cụ lập tức tìm thấy một phản ví dụ (Counterexample Trace):

$$\text{gw_idle} \xrightarrow{\text{event_packet_reg}} \text{rx_reg_packet} \\ \xrightarrow{\text{event_packet_unlock}} \text{gw_race_hazard}$$

- Tại trạng thái `gw\_race\_hazard`, nhãn `violation` xuất hiện đồng thời với việc cả hai nhãn `reg\_processing` và `unlock\_processing` cùng hoạt động, vi phạm nguyên lý Loại trừ lẫn nhau (Mutual Exclusion Violation).

- Ý nghĩa Kỹ thuật: Phân tích này chứng minh rằng việc xử lý dữ liệu mạng nhị phân bắt buộc phải có cơ chế đồng bộ hóa tuần tự (Mutex hoặc Single-threaded Event Loop), nếu không sẽ dẫn tới xung đột bộ đệm chia sẻ (Race Hazard) tại thời điểm thực thi.

6.6.3 Khai thác Toàn diện 4 Chức năng Cốt lõi của Công cụ Giảng viên (`SpecVerificationLab`)

Nhóm đã khai thác triệt để cả 4 khối chức năng tích hợp trong ứng dụng `SpecVerificationLab.exe`:

1. Chức năng 1 — Thẩm định Bảng Đặc tả & Bóc tách Ma trận (Spec Validation & Traceability):

- Ứng dụng đọc file `Nhom11\_SecureGate\_IoT\_Gateway\_Spec.xlsx`, kiểm tra cú pháp tên cột, kiểu dữ liệu của cả 10 sheets.
- Thiết lập đồ thị liên kết giữa Yêu cầu $\$to\$$ Thuộc tính kiểm chứng $\$to\$$ Kịch bản kiểm thử, đảm bảo không có yêu cầu nào bị "mồ côi" (Orphaned Requirements).

2. Chức năng 2 — Kiểm chứng Mô hình Trạng thái Hình thức (Formal Model Checking Kripke/CTL):

- Dựng ma trận kề biểu diễn đồ thị không gian trạng thái.
- Duyệt đồ thị để chứng minh hệ thống không có trạng thái bế tắc (No Deadlock).
- Đánh giá công thức CTL $AG(\neg \text{violation})$ và xuất trace phản ví dụ chỉ ra chính xác đường đi dẫn tới trạng thái lỗi `gw\_race\_hazard`.

3. Chức năng 3 — Phân tích Cú pháp Cây AST & Z3 SMT Solver:

- Trích xuất cây cú pháp trừu tượng (AST) của các hàm C trong sheet `HamSinhMa`.
- Tính toán độ phức tạp chu trình McCabe (McCabe Cyclomatic Complexity = 3) và độ sâu lồng nhau của các khối rẽ nhánh.
- Thiết lập bài toán thỏa mãn mệnh đề (SAT) với Z3 SMT Solver:
- Tìm nghiệm Checksum hợp lệ: Solver trả về SAT với giá trị mẫu `{ 'header\_checksum': 2000000001, 'payload\_checksum': 0 }`.
- Tìm nguy cơ tràn số nguyên (CWE-190): Solver tìm thấy mô hình nghiệm $\{\text{header_checksum} = 1, \text{payload_checksum} = 2,147,483,647\}$ có tổng vượt quá `INT32\_MAX` (\$2,147,483,648\$), chứng minh biểu thức cộng tích lũy bị wrap-around thành số âm.

4. Chức năng 4 — Tự động Sinh mã Kiểm toán (Code Generation) & Xuất Báo cáo Tích hợp:

- Tự động sinh mã nguồn C có gắn các tiền điều kiện và hậu điều kiện kiểm toán (`assert`).
- Tự động thực thi Fuzzing động trên mã Python sinh ra, phát hiện ngoại lệ chia cho 0 (CWE-369) và giải ngược cặp khóa bí mật (7421, 3390) sau đúng 3 vòng lặp Concolic.
- Xuất toàn bộ kết quả thành văn bản chính thức: `BaoCao\_TichHop\_SpecVerificationLab\_Nhom11.docx` (chứa 7 bảng dữ liệu và sơ đồ vector) lưu trong thư mục nộp bài.

6.6.4 Bảng Ma trận Tổng hợp 9 Phát hiện CWE/CVE từ Công cụ

Tool của giảng viên đã tự động tổng hợp toàn bộ 9 phát hiện an ninh chuẩn hóa:

Nguồn kiểm chứng	Vị trí phát hiện	Loại phát hiện kỹ thuật	Phân loại CWE chuẩn	Ví dụ CVE minh họa thực tế
Chương 1	Công thức `AG(!violation)`	`safety-property-violation`	CWE-693 (Protection Mechanism Failure)	—
Chương 1	Mô hình trạng thái Kripke	`mutual-exclusion-	CWE-667	—

		violation`	(Improper Locking)	
Chương 2	`parse_securegate_device_id`	`dangerous-function`	CWE-120 / CWE-121 (Buffer Overflow)	CVE-2003-0352 (RPC/DCOM MSBlast)
Chương 2	`log_securegate_audit_event`	`memory-leak`	CWE-401 (Missing Release of Memory)	—
Chương 2	`terminate_securegate_session`	`double-free`	CWE-415 (Double Free)	—
Chương 2	`cleanup_stream_buffer`	`use-after-free`	CWE-416 (Use After Free)	CVE-2019-0708 (BlueKeep RDP)
Chương 3	`securegate_checksum_accumulator`	`integer-overflow`	CWE-190 (Integer Overflow / Wrap)	CVE-2016-5195 (Dirty COW)
Chương 4	`calculate_gateway_drop_ratio`	`uncontrolled-crash`	CWE-369 / CWE-248 (Divide By Zero)	—
Chương 4	`verify_securegate_unlock_guard`	`uncontrolled-crash`	CWE-248 (Uncaught Exception)	—

Toàn bộ kết quả kiểm chứng đã được xuất trực tiếp thành tài liệu Word chính thức: `BaoCao\_TichHop\_SpecVerificationLab\_Nhom11.docx` lưu tại thư mục nộp bài.

Spec Verification Lab — Đặc tả → Kripke → Sinh mã → CWE/CVE

Đặc tả → Kripke → Sinh mã → CWE/CVE

Bộ đặc tả mẫu JSON (dataset/specs):

auth_rate_limit_spec.json
elevator_control_spec.json
file_upload_auth_spec.json
iot_gateway_spec.json

Bộ đặc tả Excel của nhóm (Nhom11):

elevator_control_spec.xlsx
Nhom11_SecureGate_IoT_Gateway_Spec.xlsx
resource_pool_spec.xlsx
session_manager_spec.xlsx

Nạp đặc tả Excel đã chọn

Nạp từ file Excel khác (.xlsx)...

Yêu cầu (requirements) trong đặc tả Nhóm 11:

[REQ-SG-01] Khóa Mutex luồng Registration & Unlock (Ch1)
[REQ-SG-02] Hệ thống không deadlock khi mở khóa (liveness)
[REQ-SG-03] parse_securegate_device_id() không tràn đệm CWE-120
[REQ-SG-04] Ghi log audit giải phóng bộ nhớ CWE-401
[REQ-SG-05] terminate_session() không double-free CWE-415
[REQ-SG-06] cleanup_stream_buffer() không UAF CWE-416
[REQ-SG-07] Bộ đếm checksum không tràn int32 CWE-190 (Ch3)
[REQ-SG-08] calculate_drop_ratio() không chia cho 0 CWE-369

1) Sinh Kripke + kiểm chứng (Ch1)

2) Sinh mã C/Python từ đặc tả

3) Chạy TOÀN BỘ Pipeline

Xuất báo cáo TÍCH HỢP (PDF/DOCX)

Pipeline: Đã nạp Nhom11_SecureGate & chạy 4 chức năng

Mô hình Kripke: SecureGate IoT Gateway — Kiểm chứng Thuộc tính An toàn

Mã nguồn sinh ra từ đặc tả 'securegate_iot_packet_parser' (có số dòng):

```

1  /* Sinh từ đặc tả securegate_iot_packet_parser, hàm parse_securegate_device_id */
2  void parse_securegate_device_id(const char *raw_payload_id) {
3      char buffer[16];
4      strcpy(buffer, raw_payload_id); /* LỖI: CWE-120/121 Buffer Overflow */
5      printf("%s\n", buffer);
6  }
7
8  /* Sinh từ đặc tả securegate_iot_packet_parser, hàm log_securegate_audit_event */
9  void log_securegate_audit_event(void) {
10     int *buf = malloc(sizeof(int) * 32);
11     buf[0] = 0; /* LỖI: CWE-401 Memory Leak (quên free) */
12 }
13
14 /* Sinh từ đặc tả securegate_iot_packet_parser, hàm terminate_securegate_session */
15 void terminate_securegate_session(void) {
16     int *buf = malloc(sizeof(int) * 4);
17     free(buf);
18     free(buf); /* LỖI: CWE-415 Double Free */
19 }

```

PHẦN 7 — PHẢN BIỆN & BÀI HỌC KINH NGHIỆM

7.1 Đánh giá Thiết kế và Giải pháp Ngăn ngừa từ đầu

Nếu được bắt đầu lại từ đầu dự án, nhóm sẽ áp dụng các nguyên tắc sau để triệt tiêu lỗi ngay từ giai đoạn thiết kế:

1. Áp dụng Nguyên lý Lập trình Phòng thủ (Defensive Programming):

Thay vì lưu trữ các độ dài rời rạc và dùng con trỏ C thô, nhóm sẽ đóng gói cấu trúc buffer thành một kiểu dữ liệu an toàn (Fat Pointer / Bounded Slice) chứa cả con trỏ dữ liệu và sức chứa tối đa ('capacity'). Mọi hàm thao tác chuỗi bắt buộc phải truyền 'capacity' làm tham số và tự động từ chối nếu độ dài yêu cầu vượt quá dung lượng.

2. Sử dụng Công cụ Phân tích Cú pháp Chuẩn hóa (Schema-driven Parsers):

Thay vì tự viết parser nhị phân thủ công bằng C (dễ mắc lỗi tính toán offset), nên sử dụng các framework định nghĩa giao thức chuẩn hóa (như Google Protocol Buffers, FlatBuffers hoặc ASN.1). Các công cụ này tự động sinh mã giải mã (deserializer) với cơ chế kiểm tra biên tự động và đã được kiểm chứng an toàn qua hàng triệu giờ chạy thực tế.

3. Lựa chọn Ngôn ngữ An toàn Bộ nhớ (Memory-Safe Languages):

Đối với các dịch vụ mạng nhận dữ liệu từ untrusted boundary, việc sử dụng các ngôn ngữ an toàn bộ nhớ theo khuyến nghị của NSA/CISA (như Rust) sẽ loại bỏ hoàn toàn các lớp lỗ hổng Buffer Overflow, Use-After-Free ngay tại thời điểm biên dịch (Compile-time) thông qua cơ chế Ownership và Borrow Checker.

7.2 Các Nguy cơ Bảo mật Tiềm ẩn còn Nghi ngờ

Mặc dù lỗi CWE-121 đã được vá sạch, nhóm nhận thấy parser vẫn còn một số điểm có thể tiềm ẩn rủi ro nếu mở rộng quy mô:

1. Lỗi Tràn số nguyên khi Tính Checksum (Integer Overflow / Wraparound):

Trong hàm `compute_checksum()`, phép cộng tích lũy byte được thực hiện trên kiểu `uint32_t` và ép modulo 2^{32} . Mặc dù trong chuẩn C, tràn số nguyên không dấu ('unsigned integer wraparound') là hành vi xác định rõ (Defined Behavior), nhưng nếu giao thức mở rộng với các trường độ dài lớn hơn, việc wraparound checksum có thể dẫn đến hiện tượng xung đột checksum (Collision), cho phép kẻ tấn công tạo ra hai gói tin khác nhau nhưng có cùng checksum.

2. Tấn công Từ chối Dịch vụ (Resource Exhaustion / ReDoS):

Trường `payload_len` là 16-bit cho phép gói tin có kích thước lên tới 65.535 bytes. Nếu parser được triển khai trên hệ thống nhúng có RAM cực nhỏ (ví dụ vi điều khiển chỉ có vài chục KB RAM), việc nhận liên tục các gói tin payload tối đa mà không giải phóng kịp thời có thể làm cạn kiệt bộ nhớ hệ thống (Out of Memory - OOM crash).

7.3 Bài học Phương pháp luận về Sound/Complete

Trải nghiệm thực hiện Đề tài 4 giúp nhóm nhận thức sâu sắc về hai khái niệm nền tảng trong Đảm bảo An toàn Phần mềm: Tính đúng đắn (Soundness) và Tính đầy đủ (Completeness):

- Phân tích Tĩnh (Static Analysis — Cppcheck):
- Không Sound và Không Complete đối với các thuộc tính an toàn bộ nhớ phụ thuộc runtime. Cppcheck bỏ sót lỗi CWE-121 (False Negative) vì nó không thể suy diễn được giá trị biến thiên của `hdr.device\_id\_len`. Do đó, không bao giờ được kết luận một chương trình là an toàn chỉ dựa trên việc phân tích tĩnh không báo lỗi.
- Kiểm thử Mờ Hộp đen (Black-box Fuzzing):
- Có tính Sound (nếu tìm ra crash thì chắc chắn có lỗi thực tế), nhưng cực kỳ Incomplete (không bao giờ chứng minh được chương trình không có lỗi). Black-box hoàn toàn bất lực trước các "bức tường" hằng số ma thuật (Magic Bytes / Checksum).
- Kiểm chứng Hình thức & Hộp trắng (SMT / Z3 Solving):
- Đạt tính Sound và Complete trong không gian tìm kiếm hữu hạn đã mô hình hóa: Z3 tìm ra nghiệm giải quyết ngay lập tức điều kiện equality phức tạp. Tuy nhiên, giới hạn căn bản của phương pháp này là Sự bùng nổ đường đi (Path Explosion): đối với các chương trình có hàng trăm nhánh rẽ lặp và lồng nhau, số lượng đường thi hành tăng theo cấp số nhân $O(2^n)$, khiến solver không thể scale được nếu không có các kỹ thuật cắt tỉa trạng thái.

Kết luận Phương pháp luận cốt lõi:

*Không có một công cụ đơn lẻ nào là "viên đạn bạc" giải quyết được toàn bộ bài toán an toàn phần mềm. Một quy trình kỹ thuật an toàn chuẩn mực bắt buộc phải phối hợp đa tầng: **Lập trình an toàn (Defensive Coding) $\rightarrow$ Phân tích tĩnh rà soát cú pháp $\rightarrow$ SMT Solver giải quyết các nhánh điều kiện khó $\rightarrow$ Coverage-guided Fuzzing khám phá không gian trạng thái $\rightarrow$ Runtime Sanitizers bắt lỗi thực thi**.*

PHẦN 8 — PHỤ LỤC

Phụ lục A: Hướng dẫn Cài đặt & Chạy lại (Reproduction Guide)

1. Yêu cầu Môi trường

- Trình biên dịch: GCC 11+ hoặc Clang 14+ (hỗ trợ `-fsanitize=address,undefined` và `--coverage`).
- Python: 3.10+ kèm các thư viện: `pip install -r requirements.txt` (pytest, z3-solver, matplotlib).
- Công cụ tĩnh: `cppcheck`.

2. Các bước Thực thi & Tái lập

```
# Di chuyển vào thư mục BTL
cd Nhóm11_BTL_Chude4_Fuzzing/
```

```
# Bước 1: Kiểm định tự động toàn bộ 8 khâu dự án bằng 1 lệnh
python scripts/verify_project.py
```

```
# Bước 2: Chạy bộ kiểm thử tự động 30 test cases (PyTest)
pytest -v tests/
```

```
# Bước 3: Biên dịch cả hai phiên bản vulnerable (v0) và fixed (v1) với AddressSanitizer
```

```
gcc -g -O0 -fsanitize=address,undefined source/gateway_parser_v0.c -o source/parser_vuln.exe
gcc -g -O0 -fsanitize=address,undefined source/gateway_parser_v1.c -o source/parser_fixed.exe
```

```
# Bước 4: Chạy lại toàn bộ benchmark 30 trials và sinh biểu đồ
python scripts/run_benchmark.py
python scripts/aggregate_results.py
python scripts/generate_charts.py
```

```
# Bước 5: Chạy thử nghiệm file crash trên bản fixed để xác nhận 0 finding
./source/parser_fixed.exe crashes/whitebox_s0_crash_0.bin
```

3. Quy ước Commit Chuẩn hóa (Git Commit Message)

Tuân thủ Mục 3.4 của Hướng dẫn Báo cáo Đột án:

```
fix(CWE-121): bounded copy in parse_and_process

- Replace unbounded memcpy() into 16-byte stack buffer with bound-checked copy
- Reject oversized device_id_len (> 16) instead of overflowing stack buffer
- Add regression test: test_oversized_device_id_rejected in tests/test_parser.py
- Verified: ASan clean on exploit packet (0 findings, was: 1 SEGV stack-buffer-overflow)
```

Bước 5: Chạy thử nghiệm file crash trên bản fixed để xác nhận an toàn

```
./source/parser_fixed.exe crashes/whitebox_s0_crash_0.bin
```

```
---
```

```
### Phụ lục B: Toàn văn Mã nguồn Chương trình Mục tiêu
```

```
#### 1. Mã nguồn Bản vá An toàn (`source/gateway_parser_v1.c`)
```

```
/
```

- CSE703093 - BTL An toàn phần mềm - Nhóm 11
- gateway_parser_v1.c - PHIÊN BẢN ĐÃ VÁ LỖ HỔNG AN TOÀN BỘ NHỚ (FIXED)

```
/
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <stdint.h>
```

```
#pragma pack(push, 1)
```

```
typedef struct {
```

```
char    magic[4];    / "IGW1" /
```

```
uint8_t version;    / Bước bang 1 /
```

```
uint8_t type;        / 0x01: Registration, 0x02: Unlock /
```

```

uint8_t device_id_len; / Do dai device_id trong payload /
uint8_t flags; / Bat buoc bang 0 /
uint16_t payload_len; / Do dai payload (bytes) /
uint16_t unlock_a; / Khoa mo khoa A /
uint16_t unlock_b; / Khoa mo khoa B /
uint16_t reserved; / Bat buoc bang 0 /
} GatewayHeader;

#pragma pack(pop)

#define HEADER_SIZE sizeof(GatewayHeader) / 16 bytes /
#define CHECKSUM_SIZE 4

uint32_t compute_checksum(const uint8_t data, size_t len) {
    uint32_t sum = 0;
    for (size_t i = 0; i < len; i++) {
        sum += data[i];
    }
    return sum;
}

int is_ascii_printable(const uint8_t data, size_t len) {
    for (size_t i = 0; i < len; i++) {
        if (data[i] < 0x20 || data[i] > 0x7E) return 0;
    }
    return 1;
}

int parse_and_process(const uint8_t buf, size_t file_size) {
    if (file_size < HEADER_SIZE + CHECKSUM_SIZE) {
        fprintf(stderr, "[REJECT] File qua ngan\n");
        return -1;
    }

    GatewayHeader hdr;
    memcpy(&hdr, buf, HEADER_SIZE);
    if (memcmp(hdr.magic, "IGW1", 4) != 0) {

```

```

fprintf(stderr, "[REJECT] Magic sai\n");
return -1;
}
if (hdr.version != 1) return -1;
if (hdr.flags != 0 || hdr.reserved != 0) return -1;
size_t expected_file_size = HEADER_SIZE + (size_t)hdr.payload_len + CHECKSUM_SIZE;
if (file_size != expected_file_size) return -1;
uint32_t stored_chk;
memcpy(&stored_chk, buf + HEADER_SIZE + hdr.payload_len, CHECKSUM_SIZE);
uint32_t calc_chk = compute_checksum(buf, HEADER_SIZE + hdr.payload_len);
if (stored_chk != calc_chk) {
fprintf(stderr, "[REJECT] Checksum không khớp\n");
return -1;
}
const uint8_t payload = buf + HEADER_SIZE;
if (hdr.type == 0x01) {
/ Gói tin Registration /
if (hdr.payload_len < hdr.device_id_len) return -1;
if (!is_ascii_printable(payload, hdr.device_id_len)) return -1;
/ === BẢN VÁ AN TOÀN V1: KIỂM TRA CHẶN VÀ MỞ RỘNG VÙNG ĐỆM === /
if (hdr.device_id_len > 16) {
fprintf(stderr, "[REJECT] device_id_len quá lớn: %u > 16\n", hdr.device_id_len);
return -1;
}
char device_id[17];
memcpy(device_id, payload, hdr.device_id_len);
device_id[hdr.device_id_len] = '\0';
printf("[OK] Registration: device_id=\"%s\" (%u byte), payload=%u byte\n",
device_id, hdr.device_id_len, hdr.payload_len);
return 0;
} else if (hdr.type == 0x02) {

```

```

/ Gói tin Unlock /
if (hdr.unlock_a == 7421 && hdr.unlock_b == 3390) {
printf("[OK] Unlock thanh cong!\n");
return 0;
}
return -1;
}
return -1;
}

int main(int argc, char argv[]) {
if (argc < 2) {
fprintf(stderr, "Cach dung: %s <file_goi_tin.bin>\n", argv[0]);
return 1;
}

FILE f = fopen(argv[1], "rb");
if (!f) return 1;
fseek(f, 0, SEEK_END);
long sz = ftell(f);
fseek(f, 0, SEEK_SET);
uint8_t buf = (uint8_t)malloc(sz);
if (!buf) { fclose(f); return 1; }
fread(buf, 1, sz, f);
fclose(f);
int ret = parse_and_process(buf, (size_t)sz);
free(buf);
return (ret == 0) ? 0 : 1;
}

```